Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«**МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**»

(МОСКОВСКИЙ ПОЛИТЕХ)

КАФЕДРА СМАРТ-ТЕХНОЛОГИИ

Курсовой проект по дисциплине

«**Компьютерное проектирование деталей машин**» по направлению 09.03.01 Информатика и вычислительная техника Образовательная программа (профиль)

«Разработка инженерного программного обеспечения»

Преподаватель: /_____/ Толстиков Антон Витальевич, к.т.н. /
подпись *ФИО*

Студент: /_____/ Фомичёва Эллина Дмитриевна, 241-322/
подпись *ФИО, группа*

Москва, 2026

ОГЛАВЛЕНИЕ

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ 4
2 КИНЕМАТИЧЕСКИЙ РАСЧЕТ 5
2.1 Выбор электродвигателя 5
2.2 Определение передаточного числа и параметров валов 6
2.3 Сводная таблица кинематического расчета 7
3 РАСЧЕТ ЗАКРЫТОЙ ЗУБЧАТОЙ ПЕРЕДАЧИ 8
3.1 Выбор материалов 8
3.2 Геометрический расчет 8
3.3 Проверочный расчет 9
3.4 Силы в зацеплении 10
4 РАСЧЕТ ОТКРЫТОЙ ЦЕПНОЙ ПЕРЕДАЧИ 11
4.1 Определение числа зубьев звездочек 11
4.2 Выбор шага цепи и проверка износостойкости 11
4.3 Геометрические параметры и сила на валы 11
5 РАСЧЕТ И КОНСТРУИРОВАНИЕ ВАЛОВ 13
5.1 Конструирование быстроходного вала 13
5.2 Конструирование тихоходного вала 16
6 МОДЕЛИРОВАНИЕ РЕДУКТОРА 20
6.1 Расчет элементов зубчатого колеса 20
6.2 Компоновка и корпусные элементы 22
6.3 Расчет уровня и объема масла 27
ЗАКЛЮЧЕНИЕ 28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 29

ПРИЛОЖЕНИЕ А ПРОГРАММНЫЙ МОДУЛЬ ПОСТРОЕНИЯ ЗАКЛАДНЫХ КРЫШЕК 30
ПРИЛОЖЕНИЕ Б..... 95

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Целью проекта является расчет редуктора, используемого в составе механического привода цепного рабочего органа. Исходные данные приняты согласно варианту 10-1:

- редуктор цилиндрический одноступенчатый вертикальный, прямозубый;
- диаметр звездочки D = 0,8 м;
- тяговое усилие на цепи F = 2,2 кН;
- скорость цепи V = 1,5 м/с;
- закрытая передача - цилиндрическая зубчатая;
- открытая передача - цепная;
- передаточные числа, принятые для расчета: uзп = 5 (закрытая передача), uоп = 4 (открытая передача).

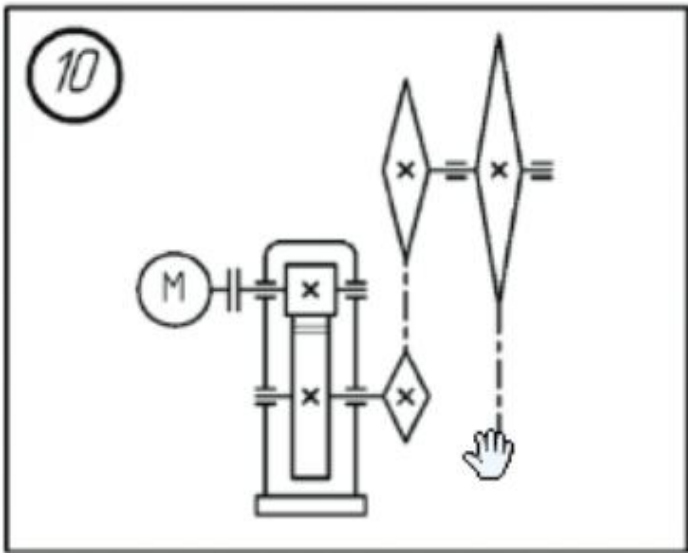


Рисунок 1 - Кинематическая схема привода по варианту 10-1

2 КИНЕМАТИЧЕСКИЙ РАСЧЕТ

2.1 Выбор электродвигателя

Мощность на валу рабочего органа определяется произведением тягового усилия на скорость цепи.

$$P_{р.в} = F \cdot V = 2{,}2 \cdot 1{,}5 = 3{,}3 \text{ кВт} \quad (2.1)$$

Общий КПД привода определяется через частные КПД закрытой передачи, открытой передачи, муфты и трех пар подшипников.

$$\eta\Sigma = \eta_{зп} \cdot \eta_{оп} \cdot \eta_{м} \cdot \eta_{пп}^3 = 0{,}96 \cdot 0{,}91 \cdot 0{,}98 \cdot 0{,}99^3 = 0{,}831 \quad (2.2)$$

$$P_{тр} = P_{р.в} / \eta\Sigma = 3{,}3 / 0{,}831 = 3{,}973 \text{ кВт} \quad (2.3)$$

Угловая скорость рабочего вала рассчитывается по диаметру звездочки и скорости цепи.

$$\omega_{р.в} = 2 \cdot V / D = 2 \cdot 1{,}5 / 0{,}8 = 3{,}75 \text{ рад/с} \quad (2.4)$$

Для предварительной разбивки передаточного числа принимаются значения $u_{зп} = 5$ и $u_{оп} = 4$. Возможное среднее рекомендуемое передаточное число привода равно:

$$u\Sigma = u_{зп} \cdot u_{оп} = 5 \cdot 4 = 20 \quad (2.5)$$

$$n_{тр.эд} = 30 \cdot \omega_{р.в} \cdot u\Sigma / \pi = 716{,}197 \text{ об/мин} \quad (2.6)$$

По требуемой частоте вращения принимается электродвигатель с фактической частотой вращения $n_{эд} = 716$ об/мин. Угловая скорость вала электродвигателя:

$$\omega_{эд} = \pi \cdot n_{эд} / 30 = \pi \cdot 716 / 30 = 74{,}979 \text{ рад/с} \quad (2.7)$$

По результатам расчета требуемая мощность электродвигателя составляет:

$P_{тр} = 3{,}973$ кВт.

Требуемая частота вращения электродвигателя определяется по формуле:

$n_{тр.эд} = (30 \cdot \omega_{р.в} \cdot u\Sigma) / \pi = 716{,}197$ об/мин.

С учетом полученных значений принимаем асинхронный электродвигатель типа АИР132S8. Номинальная мощность выбранного

двигателя составляет 4,0 кВт, что удовлетворяет условию выбора по мощности:

Рдв ≥ Ртр;

4,0 ≥ 3,973.

Выбранный электродвигатель имеет первое исполнение, номинальную мощность 4,0 кВт, 8 полюсов, частоту вращения 716 об/мин и отношение максимального момента к номинальному Tmax/Tном = 2,2.

Технические характеристики выбранного электродвигателя приведены в таблице 1.

Таблица 1 – Технические характеристики выбранного электродвигателя

Тип двигателя	**Исполнение**	**Мощность, кВт**	**Число полюсов**	**Частота вращения, об/мин**	$T_{max}/T_{\text{ном}}$
АИР132S8	1	4,0	8	716	2,2

Для установки принят двигатель первого исполнения — на лапах. Такое исполнение удобно для данного привода, так как соединение с редуктором выполняется через муфту. При этом соосность валов обеспечивается регулировкой положения двигателя на плите за счет смещения по пазам. Фланцевое исполнение в данном случае не требуется, поскольку двигатель не крепится непосредственно к корпусу редуктора.

Таким образом, выбранный электродвигатель АИР132S8 соответствует расчетным требованиям по мощности и частоте вращения и может быть использован в проектируемом приводе.

2.2 Определение передаточного числа и параметров валов

uΣ' = ωэд / ωр.в = 74,979 / 3,75 = 19,994 (2.8)

u'зп = uΣ' / uоп = 19,994 / 4 = 4,999 ≈ 5 (2.9)

u'оп = uΣ' / u'зп = 19,994 / 5 = 3,999 ≈ 4 (2.10)

Так как первый и второй валы соединены муфтой, их угловые скорости принимаются равными. Угловые скорости последующих валов определяются делением на соответствующее передаточное число.

ω1 = ωэд = 74,979 рад/с; ω2 = ω1 = 74,979 рад/с (2.11)

ω3 = ω2 / u'зп = 74,979 / 5 = 14,996 рад/с (2.12)

ω4 = ω3 / u'оп = 14,996 / 4 = 3,749 рад/с (2.13)

Проверка показывает, что ωр.в - ω4 = 0,001 рад/с, что находится в пределах допустимого расхождения.

Частоты вращения валов вычисляются по формуле n = 30ω / π. Мощность на каждом последующем валу определяется с учетом КПД соответствующего элемента привода и пары подшипников.

2.3 Сводная таблица кинематического расчета

Таблица 1 - Результаты кинематического расчета привода

Вал	**ω, рад/с**	**n, об/мин**	**P, кВт**	**T, кН·м**
1	74,979	716,0	3,973	0,053
2	74,979	716,0	3,854	0,051
3	14,996	143,2	3,663	0,244
4	3,749	35,8	3,300	0,880

Проверка моментов: T4 = P4 / ω4 = 0,88 кН·м; момент на рабочем органе Tr.в = F · D / 2 = 2,2 · 0,8 / 2 = 0,88 кН·м. Следовательно, T4 - Tr.в = 0.

3 РАСЧЕТ ЗАКРЫТОЙ ЗУБЧАТОЙ ПЕРЕДАЧИ

3.1 Выбор материалов

Закрытая передача выполнена цилиндрической прямозубой. Расчетное передаточное число закрытой передачи u12 = 5. В расчетах используются моменты на валах, переведенные в Н·мм: для шестерни T1 = 0,051 · 10^6 Н·мм, для зубчатого колеса T2 = 0,244 · 10^6 Н·мм.

$d1заг \approx 3 \cdot \sqrt[3]{(T2 / (c \cdot u12^2))} = 64{,}135$ мм (3.1)

$d2заг = d1заг \cdot u12 = 64{,}135 \cdot 5 = 320{,}676$ мм (3.2)

Таблица 2 - Материалы зубчатой передачи и допускаемые напряжения

Деталь	НВ	σHlimb, МПа	σH, МПа	σFlimb, МПа	σF, МПа
Шестерня	250	570	-	425	250
Колесо	210	490	445,455	357	210

В качестве расчетного допускаемого контактного напряжения принимается значение для зубчатого колеса: σH = 445,455 МПа.

3.2 Геометрический расчет

Поскольку расположение колес в редукторе симметричное, принимаются коэффициент нагрузки Kh = 1,2, коэффициент ширины ψbaω = 0,315 и коэффициент C = 310.

$a\omega = (uзп + 1) \cdot \sqrt[3]{((C / (\sigma H \cdot uзп))^2 \cdot T2 \cdot Kh / \psi ba\omega)} = 157{,}322$ мм (3.3)

По расчету принимается стандартное межосевое расстояние aω = 160 мм. Нормальный модуль зацепления:

$mn = 0{,}01 \cdot a\omega = 1{,}6$ мм; принято mn = 2 мм (3.4)

$zc = 2 \cdot a\omega / mn = 160$ (3.5)

$z1 = zc / (uзп + 1) = 26{,}667$; принято z1 = 27 (3.6)

$z2 = zc - z1 = 160 - 27 = 133$ (3.7)

$u' = z2 / z1 = 133 / 27 = 4{,}926$ (3.8)

Отклонение передаточного числа составляет около 1,48 %, что не превышает допустимые 3 %.

Таблица 3 - Основные геометрические параметры зубчатой передачи

Параметр	Обозначение	Значение
Межосевое расстояние	aω	160 мм
Модуль зацепления	mn	2 мм
Число зубьев шестерни	z1	27
Число зубьев колеса	z2	133
Передаточное число	u'	4,926
Делительный диаметр шестерни	d1	54 мм
Делительный диаметр колеса	d2	266 мм
Диаметр окружности выступов шестерни	da1	58 мм
Диаметр окружности выступов колеса	da2	270 мм
Диаметр окружности впадин шестерни	df1	49 мм
Диаметр окружности впадин колеса	df2	261 мм
Ширина колеса	b2	50 мм
Ширина шестерни	b1	55 мм

3.3 Проверочный расчет

Коэффициент ширины относительно диаметра и окружная скорость:

$$\psi bd = b2 / d1 = 50 / 54 = 0{,}926 \quad (3.9)$$

$$V\text{окр} = \omega 1 \cdot d1 / 2000 = 74{,}979 \cdot 54 / 2000 = 2{,}024 \text{ м/с} \quad (3.10)$$

При окружной скорости до 6 м/с принимается 8-я степень точности изготовления колес. Коэффициенты нагрузки: KHυ = 1,05; KHα = 1; KHβ = 1,04; K′H = 1,092.

$$\sigma'H = C/(a\omega \cdot u') \cdot \sqrt{(T2 \cdot K'H / b2 \cdot (u' + 1)^3)} = 414{,}426 \text{ МПа} \quad (3.11)$$

Контактная прочность обеспечена, так как σ′H = 414,426 МПа ≤ σH = 445,455 МПа.

Для проверки зубьев на изгиб приняты коэффициенты формы зуба YF1 = 3,9 и YF2 = 3,6. Сравнительная оценка показывает, что расчет следует вести по зубчатому колесу:

$\sigma F1 / YF1 = 250 / 3,9 = 64,103 > \sigma F2 / YF2 = 210 / 3,6 = 58,333$ (3.12)

Коэффициенты нагрузки при расчете на изгиб: $K F\alpha = 1$; $K F\beta = 1,1$; $K F\upsilon = 1,45$; $KF = 1,595$. Так как передача прямозубая, $Y\beta = 1$.

$\sigma F = (2 \cdot T2 \cdot KF \cdot YF2 \cdot Y\beta) / (z2 \cdot b2 \cdot mn^2) = 105,457$ МПа (3.13)

Условие прочности по напряжениям изгиба выполняется: $\sigma F = 105,457$ МПа $\leq \sigma F2 = 210$ МПа.

3.4 Силы в зацеплении

Силы в зубчатом зацеплении определяются по расчетному крутящему моменту и делительному диаметру шестерни. Для прямозубой передачи осевая сила отсутствует.

$Ft = 2 \cdot T1 / d1 = 1903,818$ Н (3.14)

$Fr = Ft \cdot tg(20°) = 692,933$ Н (3.15)

$Fn = Ft / \cos(20°) = 2026,001$ Н (3.16)

Полученные силы используются при последующем конструировании и проверке валов.

4 РАСЧЕТ ОТКРЫТОЙ ЦЕПНОЙ ПЕРЕДАЧИ

Открытая передача привода выполнена цепной. Исходные данные для расчета берутся из кинематического расчета: $T1 = T3 \cdot 10^6 = 2{,}443 \cdot 10^5$ Н·мм, $n1 = n3 = 143{,}2$ об/мин, $P1 = P3 \cdot 1000 = 3663{,}004$ Вт. Передаточное число цепной передачи $u12 = 4$.

4.1 Определение числа зубьев звездочек

$$z1 = 31 - 2 \cdot u12 + 4 = 31 - 8 + 4 = 27 \quad (4.1)$$

$$z2 = z1 \cdot u12 = 27 \cdot 4 = 108 \quad (4.2)$$

$$u'12 = z2 / z1 = 108 / 27 = 4; \quad \Delta u = 0\ \% \quad (4.3)$$

4.2 Выбор шага цепи и проверка износостойкости

Предварительно принимается среднее значение шага $t = 25{,}4$ мм и число рядов цепи $m = 1$. Для эксплуатации принимаются коэффициенты $Kд = 1{,}5$; $Ka = 1$; $Kн = 1{,}25$; $Kр = 1{,}25$; $Kсм = 1{,}5$; $Kп = 1{,}5$.

$$Kэ = Kд \cdot Ka \cdot Kн \cdot Kр \cdot Kсм \cdot Kп = 5{,}273 \quad (4.4)$$

$$p = 23 \cdot [1 + 0{,}01 \cdot (z1 - 17)] = 25{,}3 \quad (4.5)$$

$$t = 2{,}8 \cdot \sqrt[3]{(T1 \cdot Kэ / (z1 \cdot p \cdot m))} = 34{,}593 \text{ мм} \quad (4.6)$$

По расчетному значению принимается стандартный шаг цепи $t = 38{,}1$ мм.

$$V = z1 \cdot n1 \cdot t / 60000 = 27 \cdot 143{,}2 \cdot 38{,}1 / 60000 = 2{,}455 \text{ м/с} \quad (4.7)$$

$$Ft = P1 / V = 3663{,}004 / 2{,}455 = 1491{,}959 \text{ Н} \quad (4.8)$$

$$p' = pтабл \cdot [1 + 0{,}01 \cdot (z1 - 17)] = 20{,}9 \quad (4.9)$$

$$p = Ft \cdot Kэ / S = 1491{,}959 \cdot 5{,}273 / 394 = 19{,}969 \quad (4.10)$$

Условие износостойкости выполняется: $p = 19{,}969 \leq p' \cdot 1{,}05 = 21{,}945$.

4.3 Геометрические параметры и сила на валы

Предварительное межосевое расстояние принимается $a = 40 \cdot t = 1524$ мм. Расчетное число звеньев цепи:

$$Lt = 2a/t + (z1 + z2)/2 + ((z2 - z1)/(2\pi))^2 \cdot t/a = 151{,}655 \quad (4.11)$$

Принимается четное число звеньев L′t = 152. Уточненное межосевое расстояние:

$a' = 1530{,}935$ мм; $a'' = 0{,}977 \cdot a' = 1495{,}723$ мм ≈ 1496 мм (4.12)

Таблица 4 - Основные параметры цепной передачи

Параметр	**Обозначение**	**Значение**
Число зубьев ведущей звездочки	z1	27
Число зубьев ведомой звездочки	z2	108
Шаг цепи	t	38,1 мм
Скорость цепи	V	2,455 м/с
Окружная сила	Ft	1491,959 Н
Число звеньев цепи	L't	152
Уточненное межосевое расстояние	a″	1496 мм
Делительный диаметр ведущей звездочки	d1	328,19 мм
Делительный диаметр ведомой звездочки	d2	1309,97 мм
Диаметр наружной окружности ведущей звездочки	De1	347,9 мм
Диаметр наружной окружности ведомой звездочки	De2	1331,3 мм

Проверка коэффициента запаса прочности цепи:

$s = Fp / (Ft \cdot Kд + q \cdot V^2 + 9{,}81 \cdot Kf \cdot q \cdot a'' / 1000) = 52{,}975$ (4.13)

Так как $s = 52{,}975 \geq s' = 9{,}8$, запас прочности обеспечен. Сила, действующая на валы от цепной передачи:

$Fв = Ft \cdot Kв = 1491{,}959 \cdot 1{,}1 = 1641$ Н (4.14)

5 РАСЧЕТ И КОНСТРУИРОВАНИЕ ВАЛОВ

Предварительный расчет валов выполняется по пониженным допускаемым напряжениям на кручение. Для быстроходного вала используется момент ТБ = 0,051 · 10⁶ Н·мм, для тихоходного ТТ = 0,244 · 10⁶ Н·мм.

$$dБ = \sqrt[3]{(ТБ / (0,2 \cdot τкб))} = \sqrt[3]{(0,051 \cdot 10^6 / (0,2 \cdot 25))} = 21,687 \text{ мм} \quad (5.1)$$

$$dТ = \sqrt[3]{(ТТ / (0,2 \cdot τкт))} = \sqrt[3]{(0,244 \cdot 10^6 / (0,2 \cdot 15))} = 43,327 \text{ мм} \quad (5.2)$$

Все длины участков валов кроме выходных концов были приняты в соответствии с ГОСТ 6636-69, выходные концы — в соответствии с ГОСТ 12080-66. Все шпонки выбраны в соответствии с ГОСТ 23360-78.

5.1 Конструирование быстроходного вала

Быстроходный вал предназначен для передачи вращающего момента от муфты к шестерне закрытой цилиндрической передачи. Вал выполнен ступенчатым, что обеспечивает установку подшипников, шестерни и соединительных элементов. На входном конце вала предусмотрен шпоночный паз под призматическую шпонку 10×8×70, предназначенную для соединения вала с муфтой.

Диаметр входного конца быстроходного вала принят d = 32 мм, длина участка l = 80 мм. Далее предусмотрен посадочный участок под подшипник диаметром d = 35 мм и длиной l = 50 мм. В средней части вала расположена шестерня с числом зубьев Z = 27 и шириной зубчатого венца B = 55 мм. С обеих сторон шестерни предусмотрены буртики диаметром d = 45 мм и длиной l = 11 мм, обеспечивающие осевую фиксацию. Второй подшипниковый участок имеет диаметр d = 35 мм и длину l = 32 мм.

Таблица 5 - Конструктивные параметры быстроходного вала

Параметр	Обозначение	Значение
Диаметр входного конца вала	dБ	32 мм
Длина входного конца вала	lБ	80 мм
Размер шпонки на входном	b × h × l	10 × 8 × 70 мм

Параметр	Обозначение	Значение
конце		
Диаметр левого подшипникового участка	dпл	35 мм
Длина левого подшипникового участка	lпл	50 мм
Диаметр буртика перед шестерней	dб1	45 мм
Длина буртика перед шестерней	lб1	11 мм
Число зубьев шестерни	Z1	27
Диаметр делительной окружности шестерни	d1	54 мм
Ширина шестерни	b1	55 мм
Диаметр буртика после шестерни	dб2	45 мм
Длина буртика после шестерни	lб2	11 мм
Диаметр правого подшипникового участка	dпп	35 мм

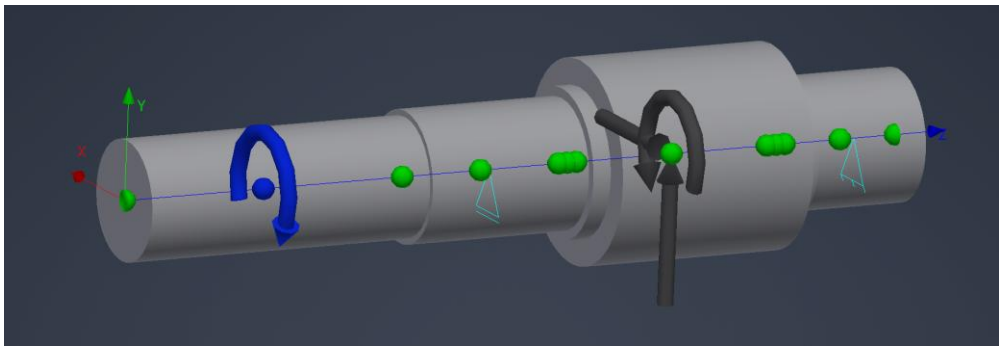


Рисунок 2 – 3D модель быстроходного вала с расставленными силами в Autodesk Inventor

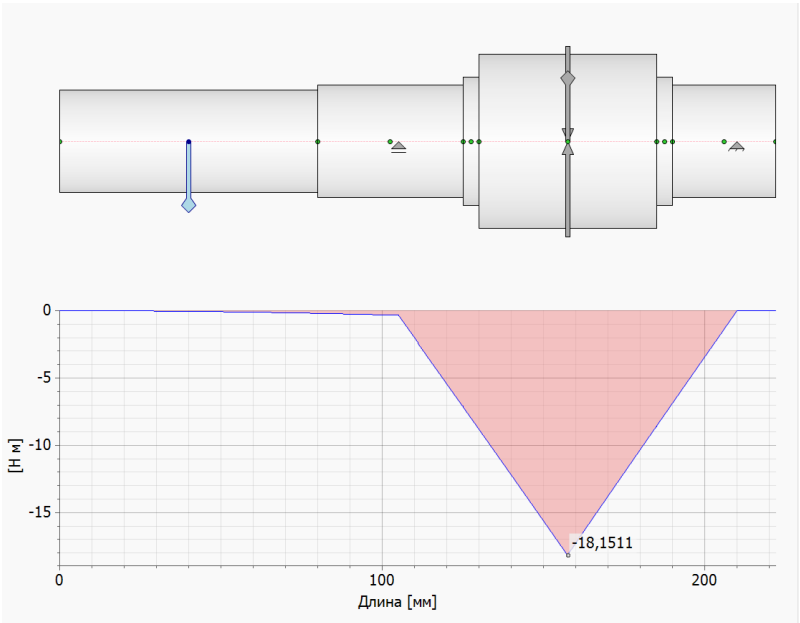


Рисунок 3 - Эпюра изгибающего момента плоскости Yz быстроходного вала в Autodesk Inventor

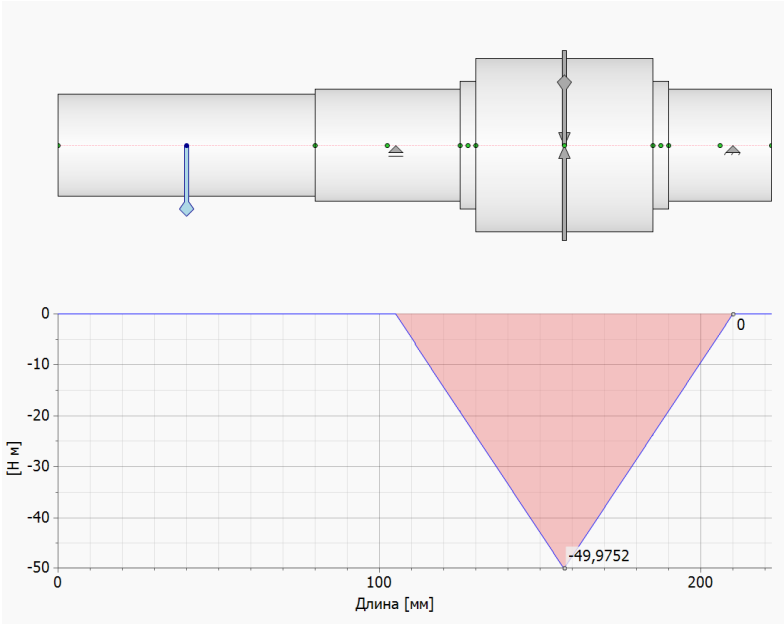


Рисунок 4 - Эпюра изгибающего момента плоскости Xz быстроходного вала в Autodesk Inventor

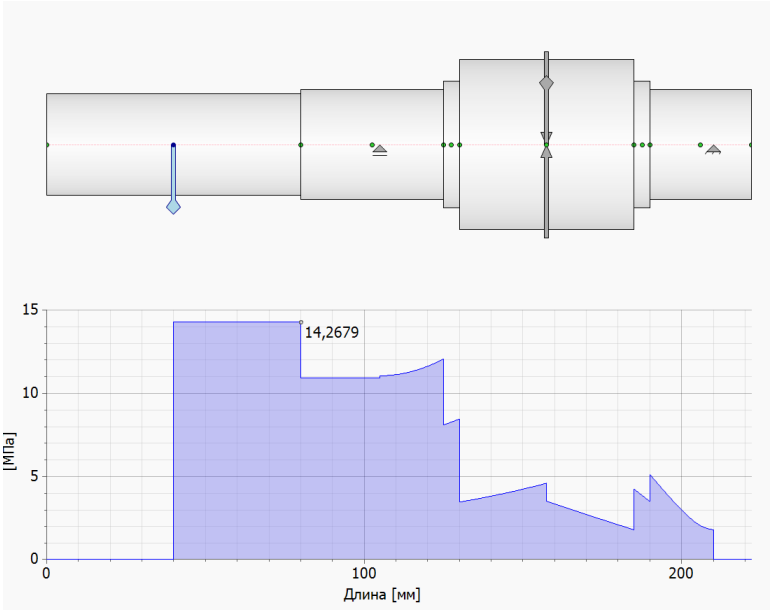


Рисунок 5 - Эпюра приведенного напряжения быстроходного вала в Autodesk Inventor

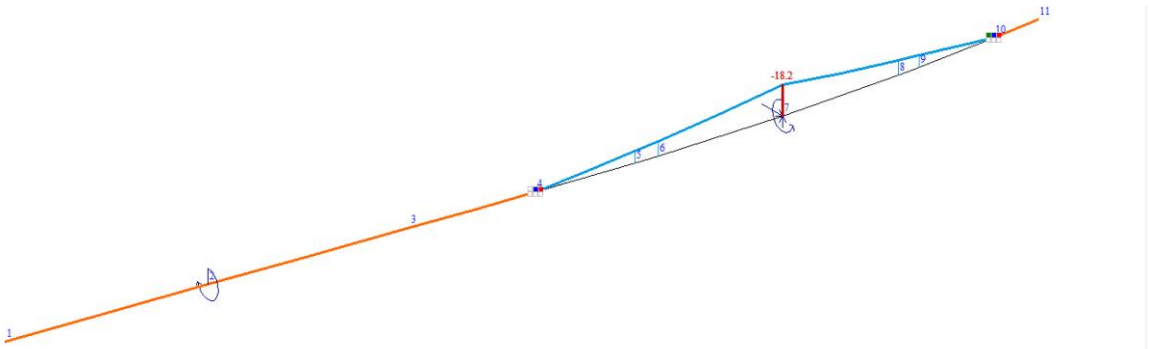


Рисунок 6 - Эпюра изгибающего момента Мх быстроходного вала в ЛИРА-САПР

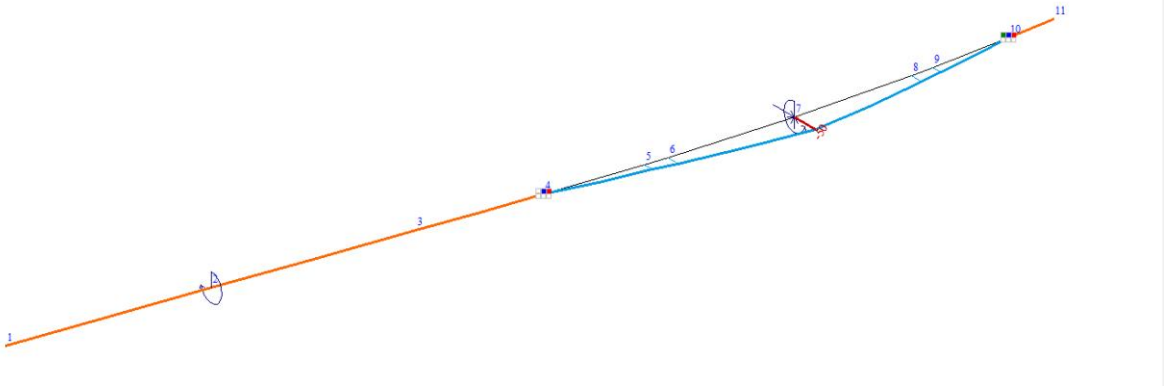


Рисунок 7 - Эпюра изгибающего момента Му быстроходного вала в ЛИРА-САПР

5.2 Конструирование тихоходного вала

Тихоходный вал предназначен для передачи вращающего момента от зубчатого колеса закрытой передачи к выходному элементу привода. Вал выполнен ступенчатым, с посадочными поверхностями под подшипники, зубчатое колесо и выходной конец. Для передачи крутящего момента предусмотрены шпоночные пазы под призматические шпонки.

На одном из участков тихоходного вала предусмотрен шпоночный паз под шпонку 16×10×45, предназначенный для установки зубчатого колеса. На выходном конце вала выполнен шпоночный паз под шпонку 14×9×100, предназначенный для соединения с выходным элементом привода.

Первый посадочный участок имеет диаметр d = 50 мм и длину l = 50 мм. Далее расположен участок диаметром d = 55 мм и длиной l = 48 мм со шпоночным пазом под шпонку 16×10×45. В средней части вала предусмотрен буртик диаметром d = 60 мм и длиной l = 14 мм. Следующий участок имеет диаметр d = 50 мм и длину l = 67 мм. Выходной конец вала

выполнен диаметром d = 45 мм и длиной l = 110 мм, на нем предусмотрен шпоночный паз под шпонку 14×9×100.

Таблица 6 - Конструктивные параметры тихоходного вала

Параметр	Обозначение	Значение
Диаметр левого подшипникового участка	dпл	50 мм
Длина левого подшипникового участка	lпл	50 мм
Диаметр посадочного участка под зубчатое колесо	dк	55 мм
Длина посадочного участка под зубчатое колесо	lк	48 мм
Размер шпонки под зубчатое колесо	b × h × l	16 × 10 × 45 мм
Диаметр буртика	dб	60 мм
Длина буртика	lб	14 мм
Диаметр правого подшипникового участка	dпп	50 мм
Длина правого подшипникового участка	lпп	67 мм
Диаметр выходного конца вала	dT	45 мм
Длина выходного конца вала	lT	110 мм
Размер шпонки на выходном конце	b × h × l	14 × 9 × 100 мм
Диаметр делительной окружности зубчатого колеса	d2	266 мм
Ширина зубчатого колеса	b2	48 мм
Окружная сила в зацеплении	Ft2	1903,818 Н
Радиальная сила в зацеплении	Fr2	692,933 Н
Нормальная сила в зацеплении	Fn2	2026,001 Н

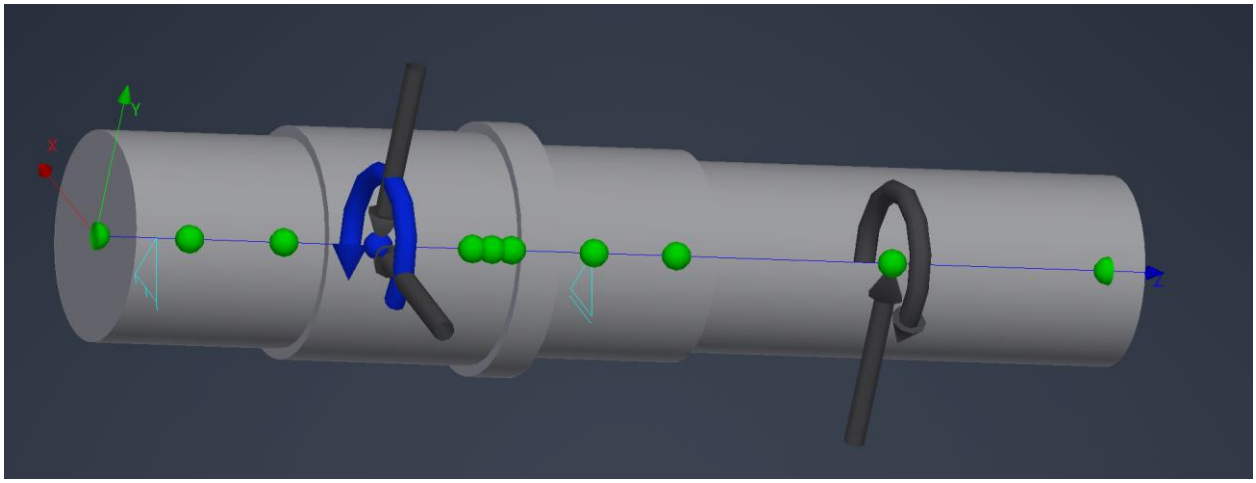


Рисунок 8 - 3D модель тихоходного вала с расставленными силами в Autodesk Inventor

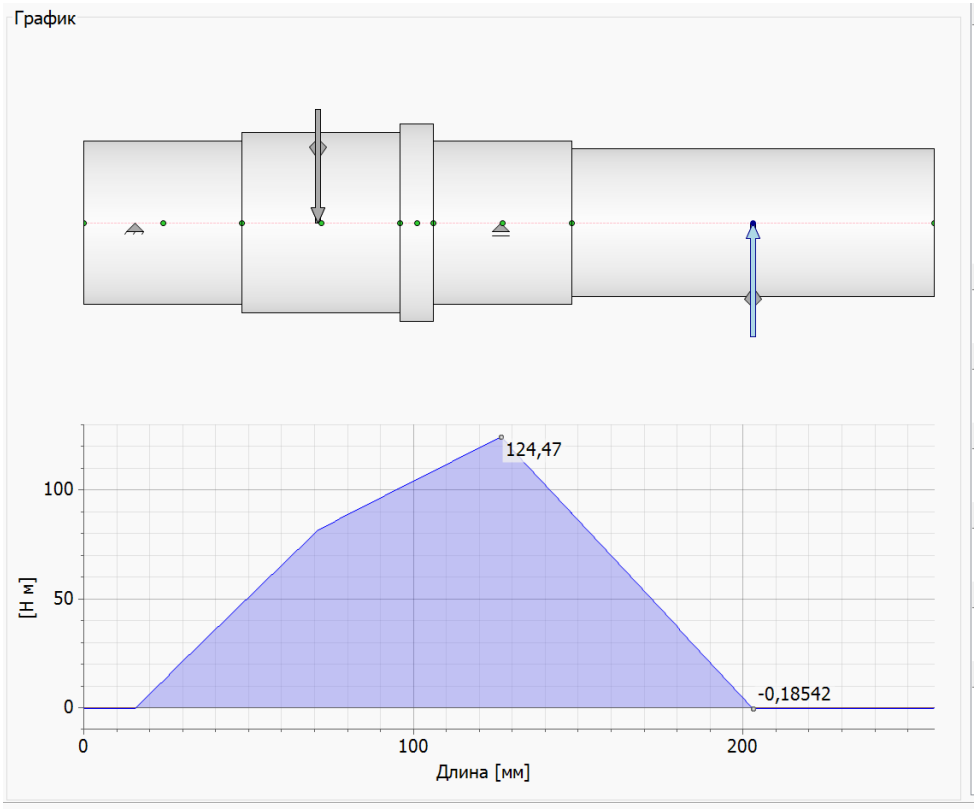


Рисунок 9 - Эпюра изгибающего момента плоскости Yz быстроходного вала в Autodesk Inventor

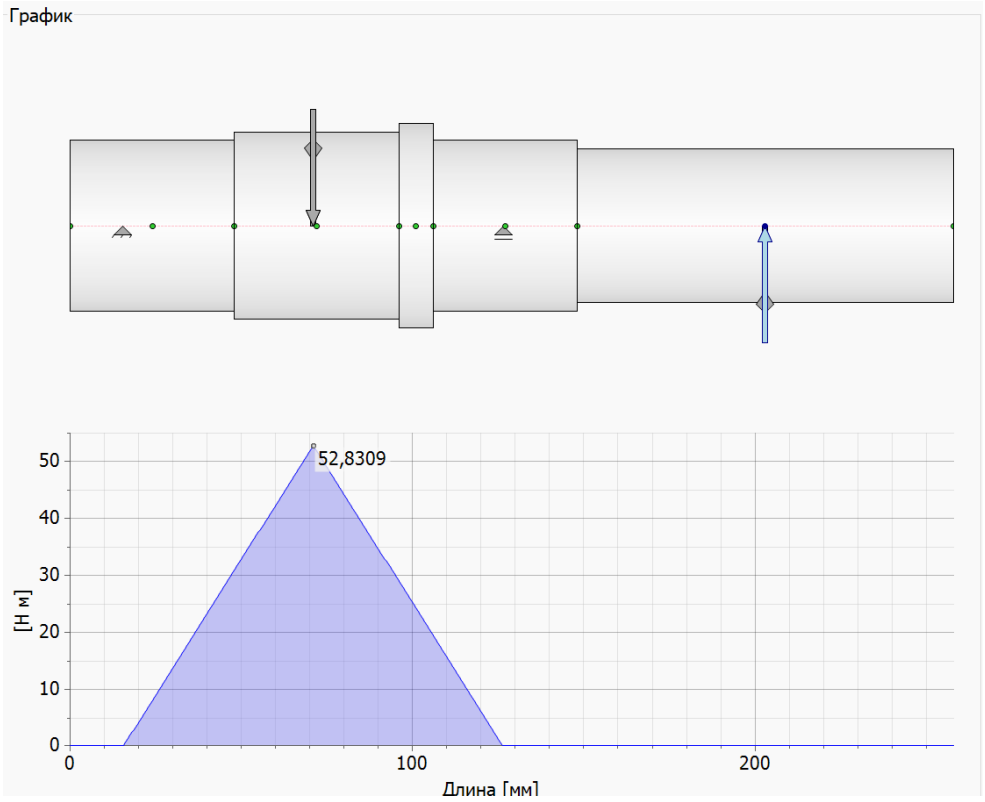


Рисунок 10 - Эпюра изгибающего момента плоскости Xz тихоходного вала в Autodesk Inventor

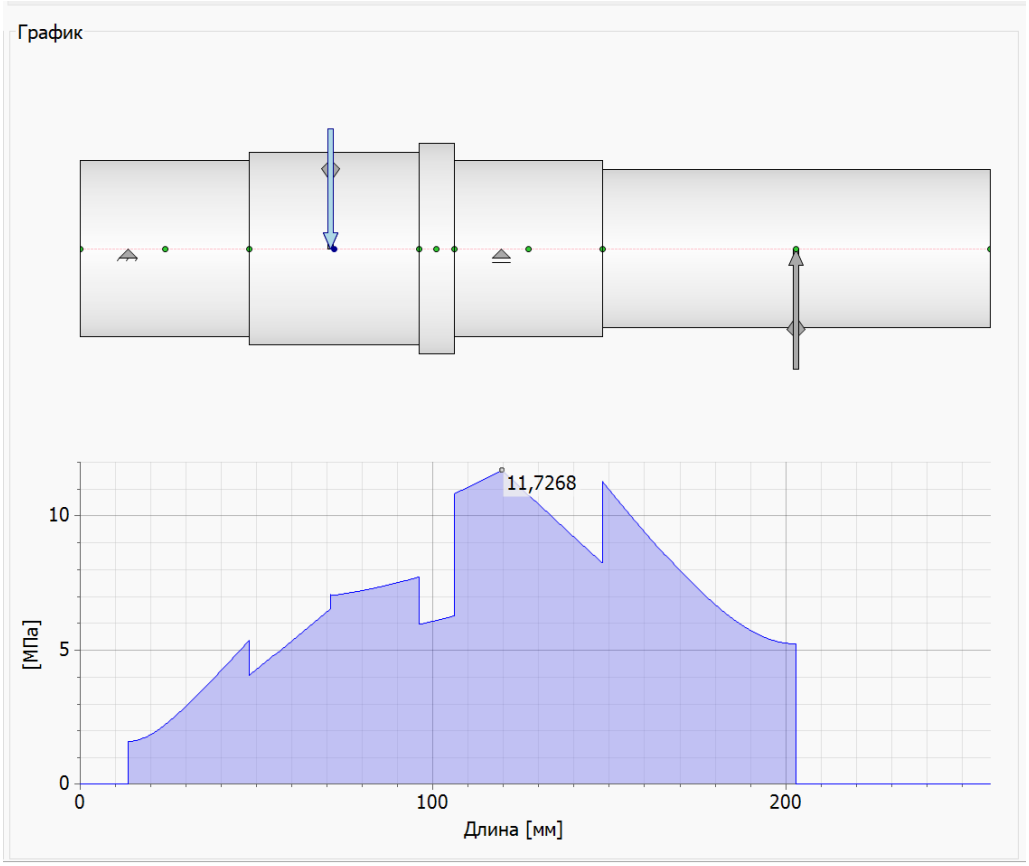


Рисунок 11 - Эпюра приведённого напряжения тихоходного вала в Autodesk Inventor

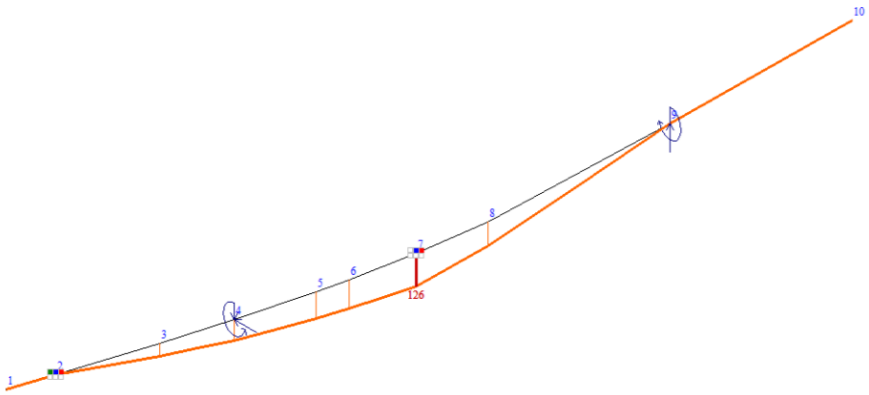


Рисунок 12 - Эпюра изгибающего момента Мх быстроходного вала в ЛИРА-САПР

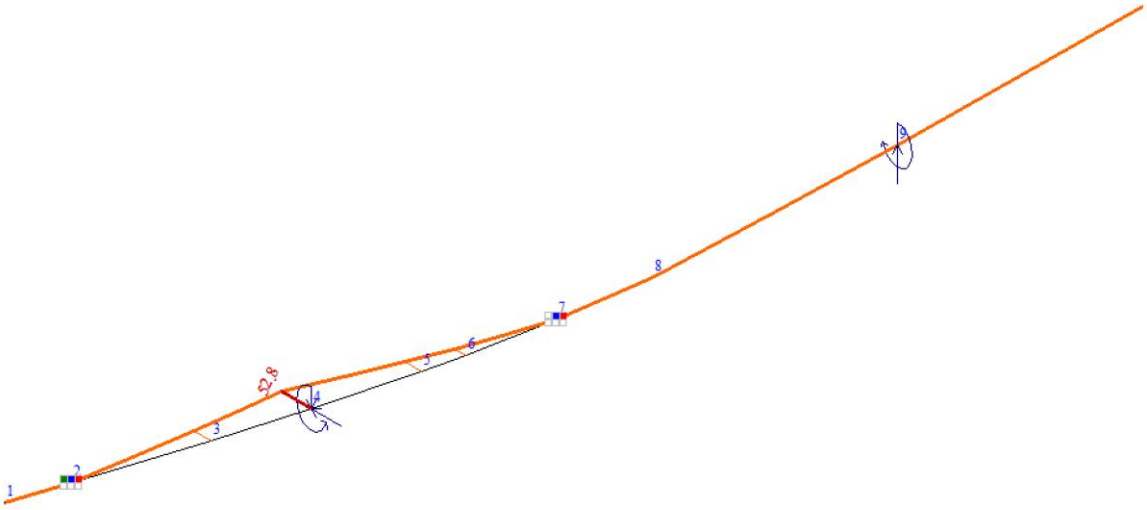


Рисунок 13 - Эпюра изгибающего момента My быстроходного вала в ЛИРА-САПР

6 МОДЕЛИРОВАНИЕ РЕДУКТОРА

При компоновке редуктора принимается расстояние от боковых поверхностей шестерни и выступов зубчатого колеса до внутренней стенки корпуса:

$\Delta = 1{,}5 \cdot 8 = 12$ мм (6.1), где 8 — толщина стенки копуса.

6.1 Расчет элементов зубчатого колеса

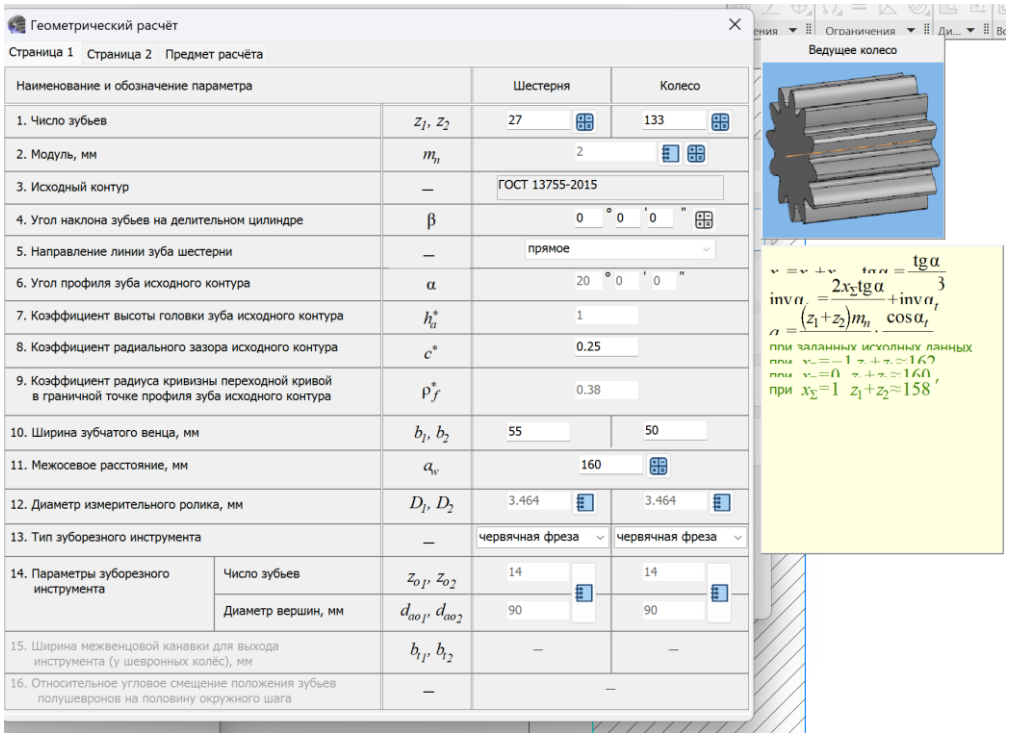


Рисунок 14 – Расчет зубчатой передачи в КОМПАС-3D

Для зубчатого колеса выполняется расчет отверстий, ступицы и диска. При модуле mn = 2 мм толщина торца принимается Stop = 4 · mn = 8 мм.

$D0 = d2 - 2 \cdot Stop = 266 - 16 = 250$ мм (6.2)

$dст = 1{,}6 \cdot 55 = 88$ мм; $lст = 50$ мм (6.3)

$Dотв = 0{,}5 \cdot (D0 + dст) = 0{,}5 \cdot (250 + 88) = 169$ мм (6.4)

$dотв = (D0 - dст) / 4 = (250 - 88) / 4 = 40{,}5$ мм (6.5)

$Сдиск = 0{,}3 \cdot 50 = 15$ мм (6.6)

Фаска на колесе со стороны зубьев принимается $f = 0{,}5 \cdot mn = 1$ мм под углом 45°.

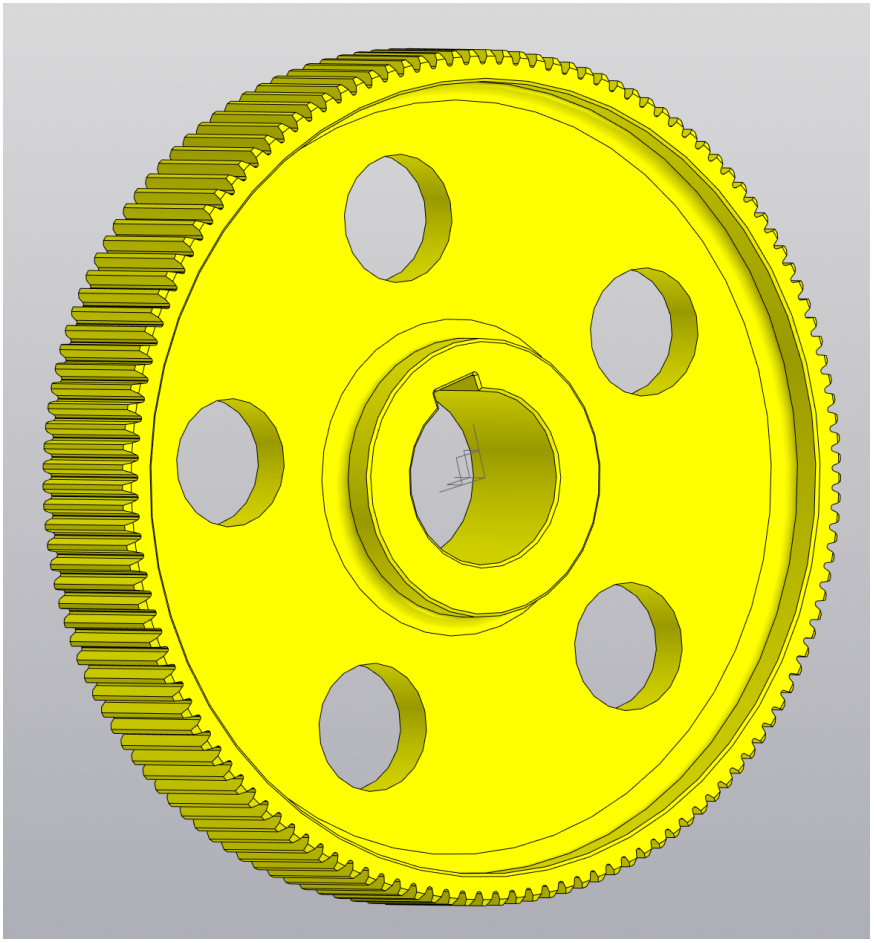


Рисунок 15 - 3D-модель зубчатого колеса

6.2 Компоновка и корпусные элементы

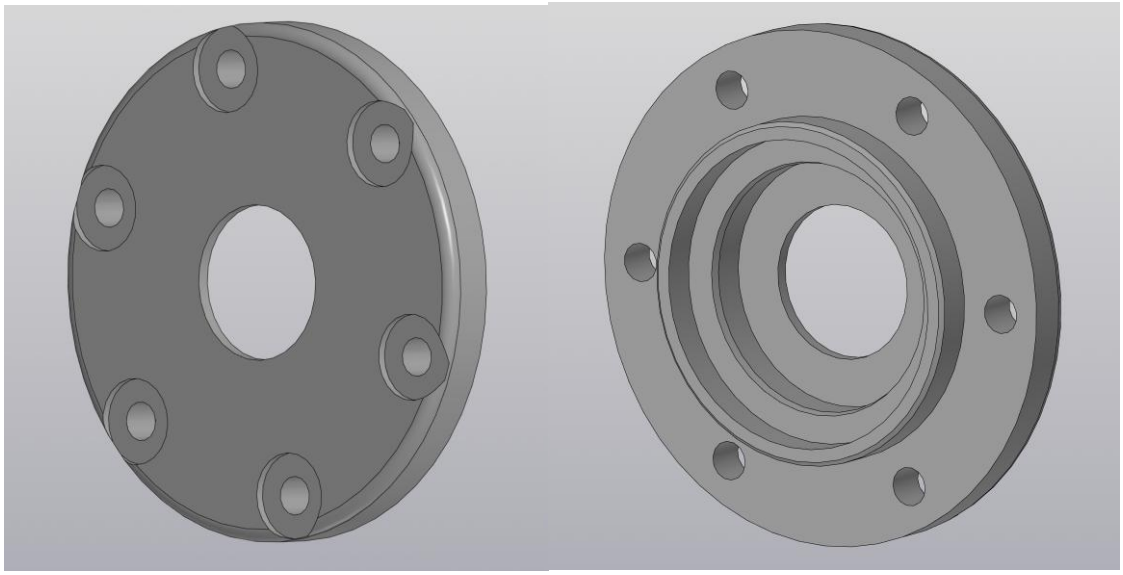


Рисунок 16 - Модель сквозной крышки редуктора

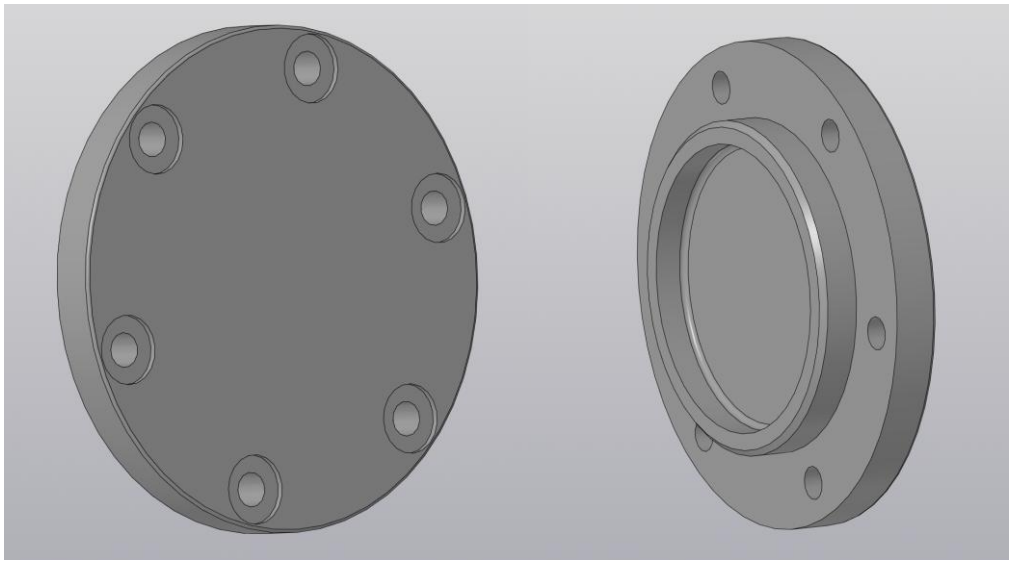


Рисунок 17 - Модель глухой крышки редуктора

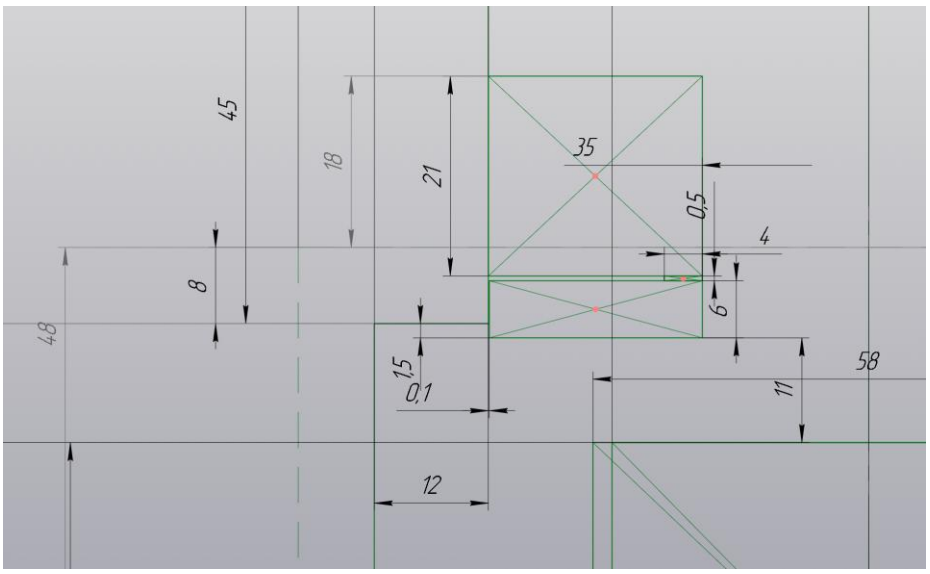


Рисунок 18 – Компоновка для быстроходного вала

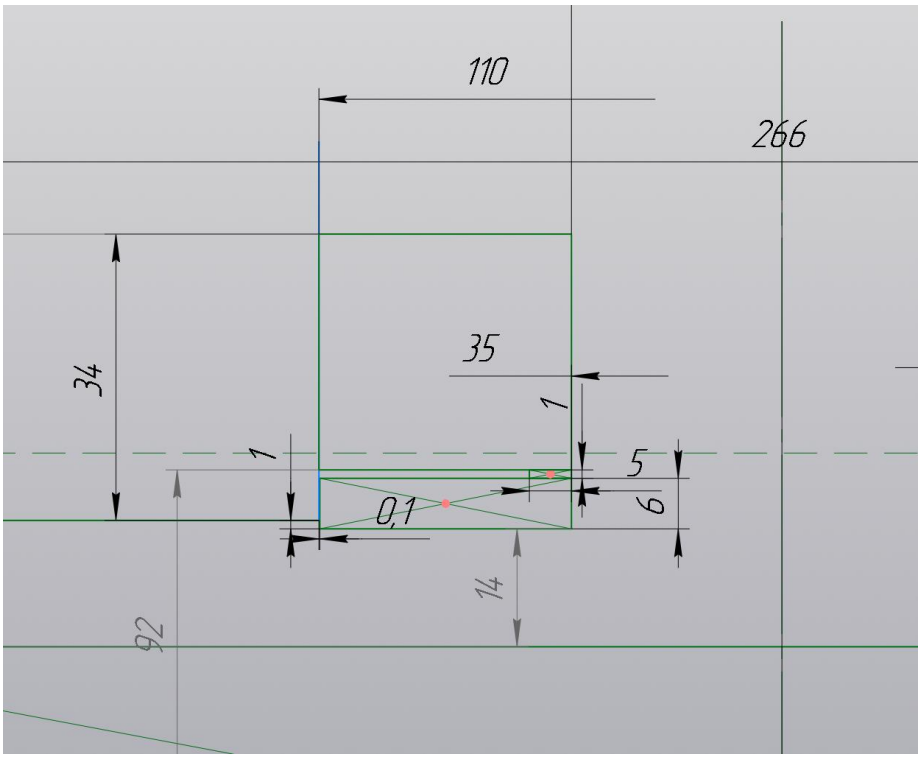


Рисунок 19 – Компоновка для тихоходного вала

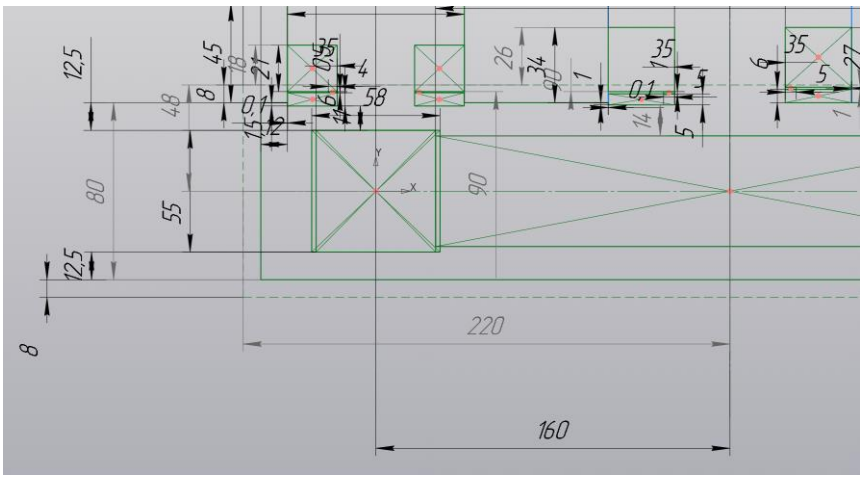


Рисунок 20 – Главный вид компоновки

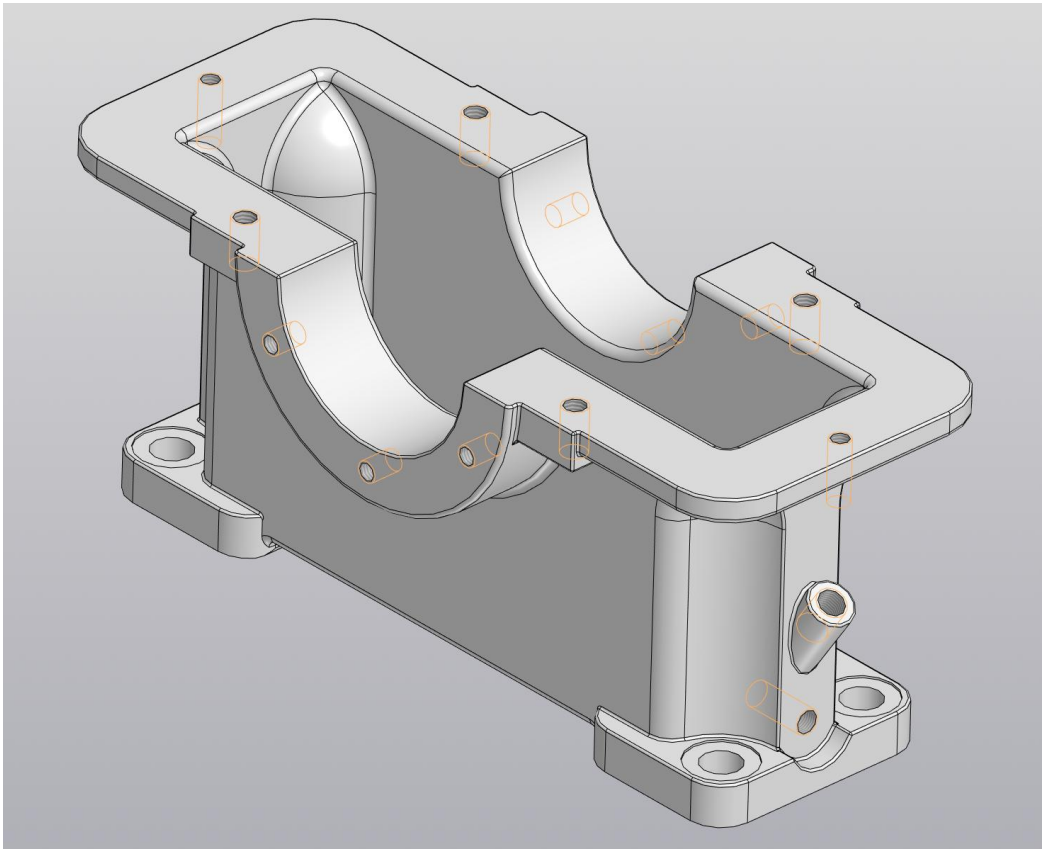


Рисунок 21 – Основание корпуса

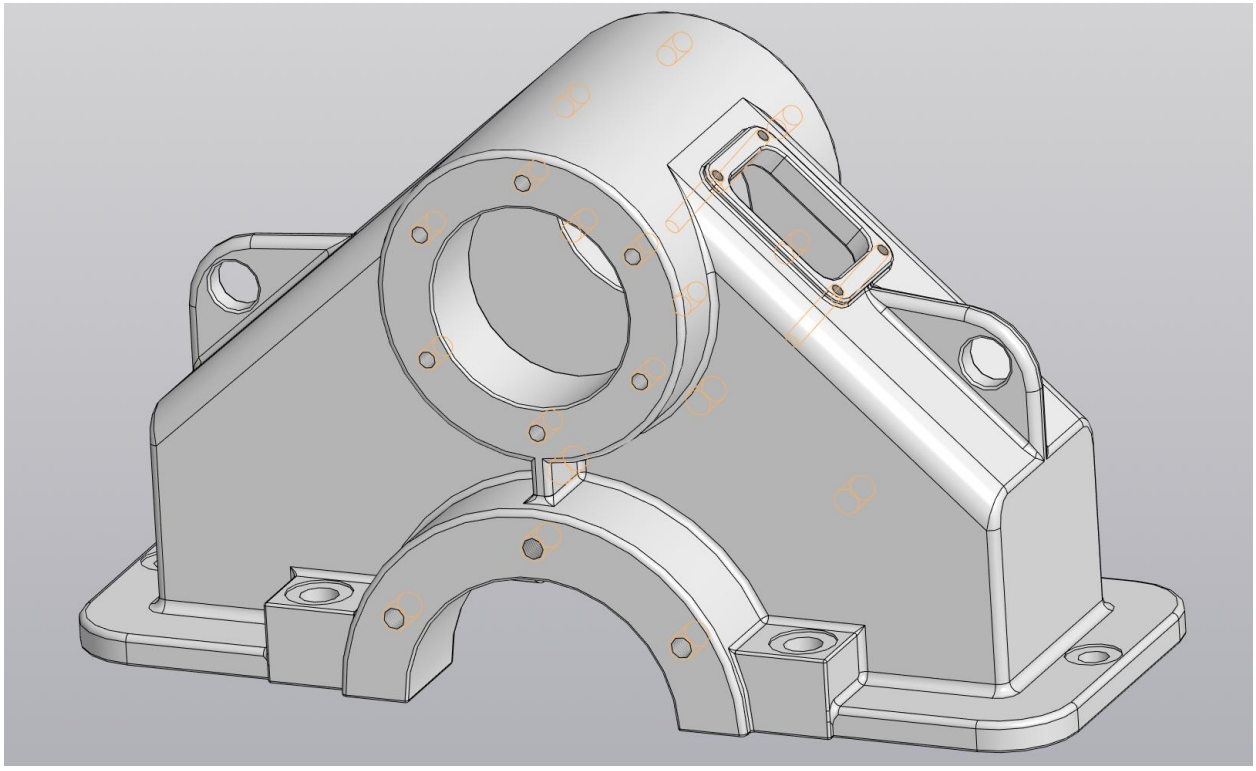


Рисунок 22 – Крышка корпуса

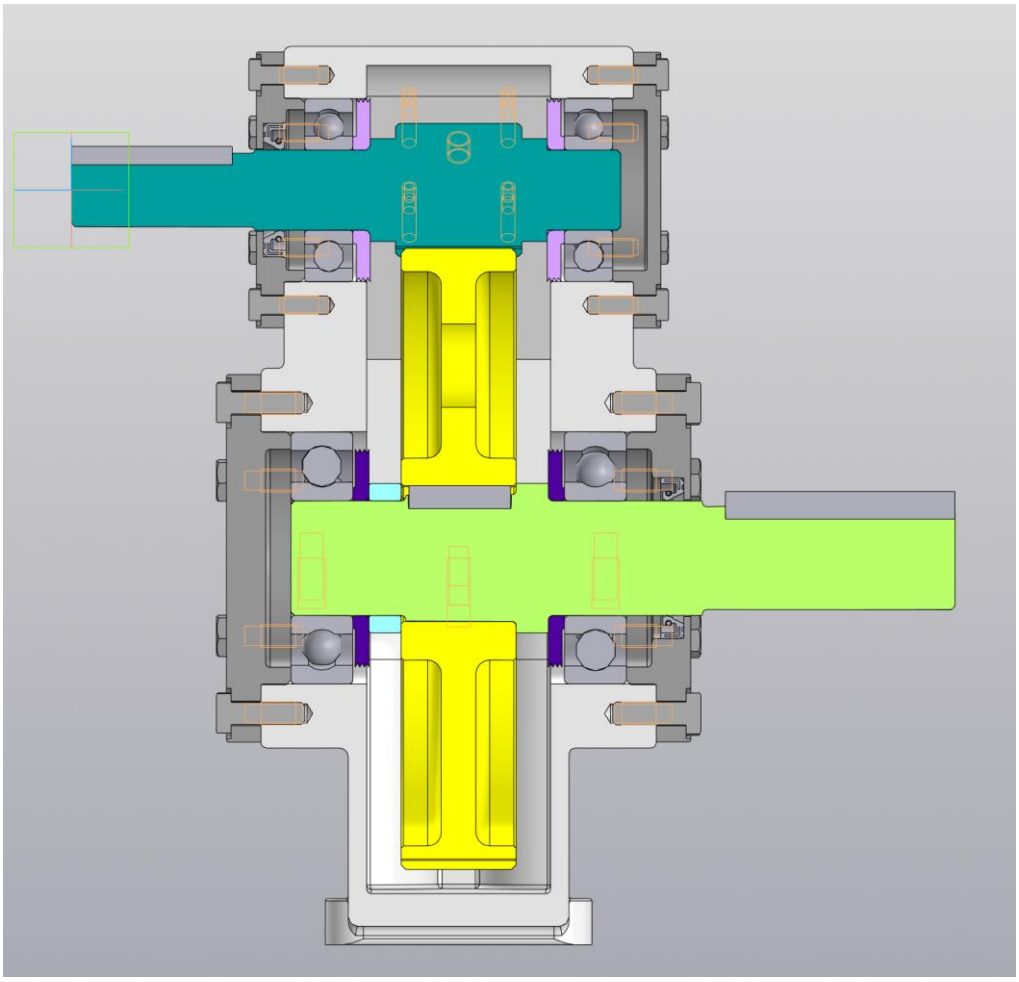


Рисунок 23 – Редуктор в собранном виде, поперечный разрез

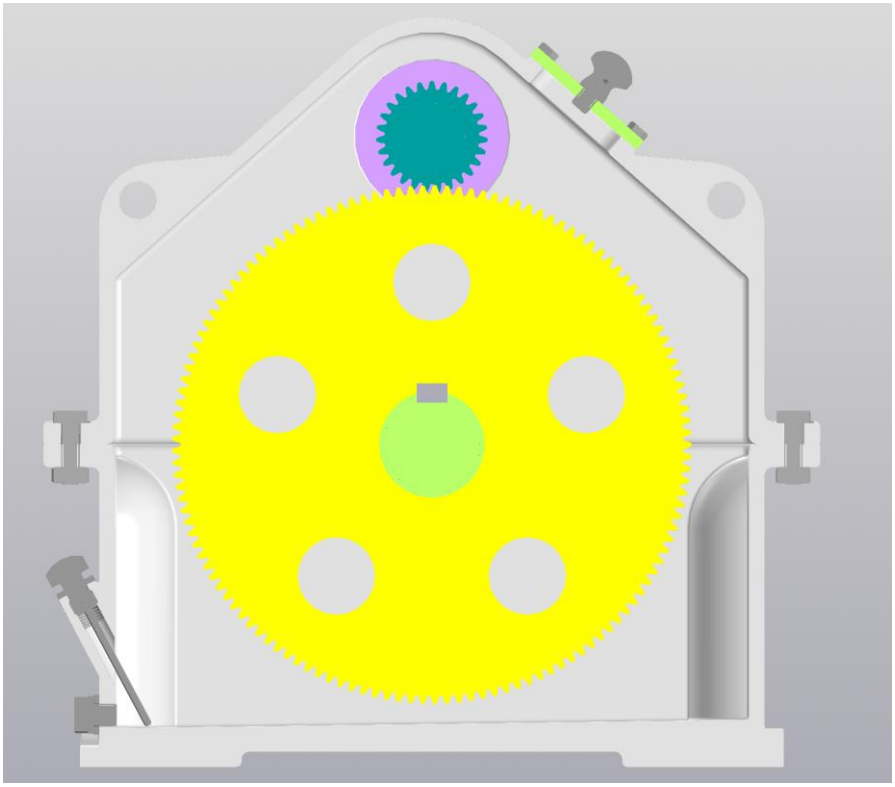


Рисунок 24 – Редуктор в собранном виде, продольный разрез

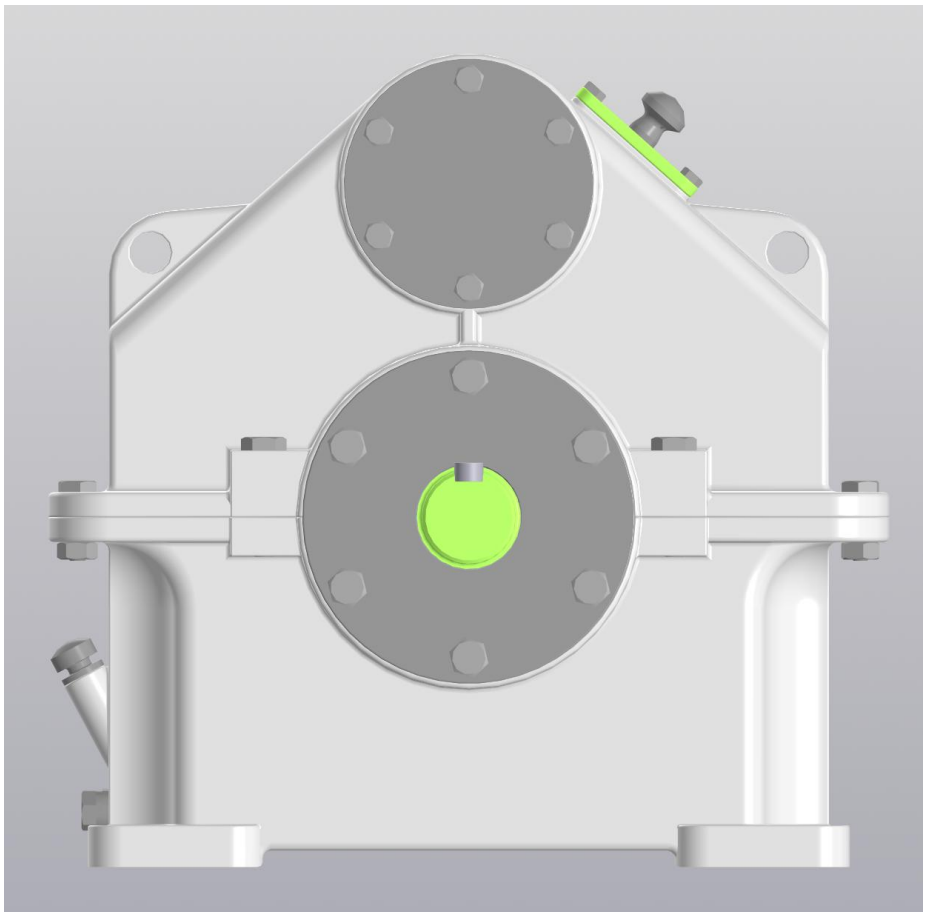


Рисунок 25 – Редуктор в собранном виде

6.3 Расчет уровня и объема масла

Минимальный и максимальный уровни масла определяются по модулю зацепления:

$h_{min} = m_n \cdot 1 + 10 = 2 \cdot 1 + 10 = 12$ мм (6.7)

$h_{max} = h_{min} + m_n \cdot 5 = 12 + 2 \cdot 5 = 22$ мм (6.8)

Площадь ванны редуктора по расчету составляет 23590 мм². Объем масла:

$V_{масла} = S_{ванны} \cdot h_{max} = 23590 \cdot 22 = 518980$ мм³ = 0,519 л (6.9)

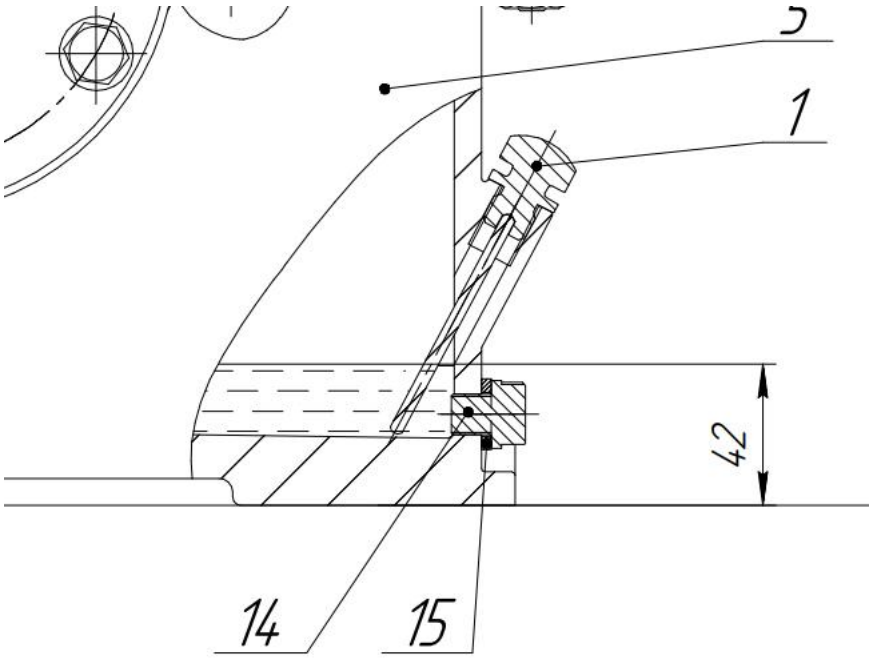


Рисунок 26 – Уровень масла в редукторе на чертеже

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта выполнен расчет цилиндрического одноступенчатого вертикального редуктора с прямозубой закрытой передачей и открытой цепной передачей. По исходным данным варианта 10-1 получена мощность рабочего органа $P_{р.в} = 3{,}3$ кВт и требуемая мощность электродвигателя $P_{тр} = 3{,}973$ кВт. Принято общее передаточное число $u_\Sigma \approx 20$ с разбивкой между закрытой и открытой передачами 5 и 4 соответственно.

В результате геометрического расчета закрытой передачи приняты: межосевое расстояние $a_\omega = 160$ мм, модуль $m_n = 2$ мм, число зубьев $z_1 = 27$ и $z_2 = 133$, ширины зубчатых венцов $b_1 = 55$ мм и $b_2 = 50$ мм. Проверочные расчеты показали выполнение условий контактной прочности и прочности на изгиб.

Для открытой цепной передачи приняты звездочки $z_1 = 27$ и $z_2 = 108$, шаг цепи $t = 38{,}1$ мм, уточненное межосевое расстояние $a'' = 1496$ мм. Коэффициент запаса прочности цепи $s = 52{,}975$ превышает требуемое значение $s' = 9{,}8$.

По предварительному расчету приняты диаметры быстроходного и тихоходного валов 32 мм и 45 мм. Также выполнены расчет элементов зубчатого колеса и расчет объема масла в корпусе редуктора, равный 0,519 л. В документе оставлены подписанные места для вставки скриншотов из расчетных и CAD-приложений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чернавский, С. А. Курсовое проектирование деталей машин: учебное пособие / С. А. Чернавский, К. Н. Боков, И. М. Чернин и др. - 3-е изд., перераб. и доп. - М.: ИНФРА-М, 2014. - 414 с.

2. Дунаев, П. Ф. Конструирование узлов и деталей машин: учебное пособие / П. Ф. Дунаев, О. П. Леликов. - 8-е изд., перераб. и доп. - М.: Академия, 2004. - 496 с.

3. Дунаев, П. Ф. Детали машин. Курсовое проектирование: учебное пособие / П. Ф. Дунаев, О. П. Леликов. - 5-е изд., доп. - М.: Машиностроение, 2004. - 560 с.

4. Шейнблит, А. Е. Курсовое проектирование деталей машин: учебное пособие. - 2-е изд., перераб. и доп. - Калининград: Янтарный сказ, 2002. - 454 с.

5. Чехович, Л. И. Атлас конструкций редукторов: учебное пособие / Л. И. Чехович, И. П. Петриниченко. - 2-е изд., перераб. и доп. - Киев: Вища школа, 1990. - 150 с.

6. Решетов, Д. Н. Детали машин: атлас конструкций: учебное пособие / Д. Н. Решетов, В. Н. Богачев, А. В. Бужинке и др. - 5-е изд., перераб. и доп. - М.: Машиностроение, 1992. - 352 с.

7. ГОСТ 2185-66. Передачи зубчатые цилиндрические. Основные параметры. - М.: Издательство стандартов, 1967. - 12 с.

8. ГОСТ 9563-60. Основные нормы взаимозаменяемости. Колеса зубчатые. Модули. - М.: Издательство стандартов, 1960. - 8 с.

9. ГОСТ 21354-87. Передачи зубчатые цилиндрические эвольвентные внешнего зацепления. Расчет на прочность. - М.: Издательство стандартов, 1987.

10. Методические указания к выполнению курсового проекта по дисциплине «Инженерное проектирование». - М.: Московский Политех, 2022.

ПРИЛОЖЕНИЕ А ПРОГРАММНЫЙ МОДУЛЬ ПОСТРОЕНИЯ ЗАКЛАДНЫХ КРЫШЕК

```
## Файл `targetver.h`
```cpp
#pragma once
#include <SDKDDKVer.h>
## Файл `framework.h`
```cpp
#pragma once
#ifndef VC_EXTRALEAN
#define VC_EXTRALEAN
#endif
#include "targetver.h"
#define _ATL_CSTRING_EXPLICIT_CONSTRUCTORS
#define _AFX_ALL_WARNINGS
#include <afxwin.h>
#include <afxext.h>
#include <afxdisp.h>
#ifndef _AFX_NO_OLE_SUPPORT
#include <afxdtctl.h>
#endif
#ifndef _AFX_NO_AFXCMN_SUPPORT
#include <afxcmn.h>
#endif
#include <afxcontrolbars.h>
#ifdef _UNICODE
#if defined _M_IX86
#pragma comment(linker,"/manifestdependency:\"type='win32'
name='Microsoft.Windows.Common-Controls' version='6.0.0.0'
processorArchitecture='x86' publicKeyToken='6595b64144ccf1df'
language='*'\"")
#elif defined _M_X64
#pragma comment(linker,"/manifestdependency:\"type='win32'
```

```
name='Microsoft.Windows.Common-Controls' version='6.0.0.0'
processorArchitecture='amd64' publicKeyToken='6595b64144ccf1df'
language='*'\"")
#else
#pragma comment(linker,"/manifestdependency:\"type='win32'
name='Microsoft.Windows.Common-Controls' version='6.0.0.0'
processorArchitecture='*' publicKeyToken='6595b64144ccf1df'
language='*'\"")
#endif
#endif
```

Файл `pch.h`

```cpp
#ifndef PCH_H
#define PCH_H
#include "framework.h"
#include"D:\Program Files\ASCON\KOMPAS-3D v24
Study\SDK\Include\ksConstants.h"
#include"D:\Program Files\ASCON\KOMPAS-3D v24
Study\SDK\Include\ksConstants3D.h"
#include"D:\Program Files\ASCON\KOMPAS-3D v24
Study\SDK\Include\kAPI7.h"
#import"D:\Program Files\ASCON\KOMPAS-3D v24 Study\SDK\lib\kAPI5.tlb"
\
    rename("min", "kAPI5_min") \
    rename("max", "kAPI5_max") \
    rename("GetObject", "kAPI5_GetObject")
#import"D:\Program Files\ASCON\KOMPAS-3D v24 Study\SDK\lib\kAPI7.tlb"
\
    rename("DeleteFile", "kAPI7_DeleteFile") \
    rename("PostMessage", "kAPI7_PostMessage") \
    rename("GetObject", "kAPI7_GetObject") \
```

```
    rename("FindText", "kAPI7_FindText") \
    rename("MessageBoxEx", "kAPI7_MessageBoxEx") \
    rename("DrawText", "kAPI7_DrawText")
#endif
```

Файл `pch.cpp`

```cpp
#include "pch.h"
```

Файл `Resource.h`

```cpp
#define IDR_MAINFRAME 128
#define IDM_ABOUTBOX 0x0010
#define IDD_ABOUTBOX 100
#define IDS_ABOUTBOX 101
#define IDD_KURSOVAYA_DETMASH_DIALOG 102
#define IDC_RADIO_OVERLAY 1000
#define IDC_RADIO_INSET 1001
#define IDC_EDIT_D 1002
#define IDC_EDIT_H 1003
#define IDC_STATIC_H_RANGE 1004
#define IDC_CHECK_SEAL 1005
#define IDC_RADIO_SEAL_TYPE1 1006
#define IDC_RADIO_SEAL_TYPE2 1007
#define IDC_COMBO_SHAFT_D 1008
#define IDC_EDIT_D1_RESULT 1010
#define IDC_EDIT_D2_RESULT 1011
#define IDC_EDIT_D3_RESULT 1012
#define IDC_EDIT_H_RESULT 1013
```

```
#define IDC_EDIT_H_SMALL_RESULT 1014
#define IDC_EDIT_S_RESULT 1015
#define IDC_EDIT_B_RESULT 1016
#define IDC_EDIT_HOLE_D_RESULT 1017
#define IDC_EDIT_COUNTERBORE_RESULT 1018
#define IDC_EDIT_THREAD_RESULT 1019
#define IDC_EDIT_HOLE_COUNT_RESULT 1020
#define IDC_EDIT_SEAL_D_RESULT 1021
#define IDC_EDIT_SEAL_H_RESULT 1022
#define IDC_EDIT_CLEARANCE_RESULT 1023
#define IDC_STATIC_STATUS 1024
#define IDC_BUTTON_BUILD 1025
#define IDC_BUTTON_RESET 1026
#define IDC_EDIT_FILLET_RESULT 1027
#define IDC_EDIT_OUTER_FILLET_RESULT 1028
#define IDC_TAB_MAIN 1029
#define IDC_STATIC_DRAWING_OVERLAY 1030
#define IDC_STATIC_DRAWING_INSET 1031
#define IDC_STATIC_DRAWING_SEALS 1032
#define IDC_STATIC_DRAWING_OVERLAY_HINT 1033
#define IDC_STATIC_DRAWING_INSET_HINT 1034
#define IDC_STATIC_DRAWING_SEALS_HINT 1035
#ifdef APSTUDIO_INVOKED
#ifndef APSTUDIO_READONLY_SYMBOLS
#define _APS_NEXT_RESOURCE_VALUE 129
#define _APS_NEXT_CONTROL_VALUE 1036
#define _APS_NEXT_SYMED_VALUE 101
#define _APS_NEXT_COMMAND_VALUE 32771
#endif
#endif
```

Файл `CoverParameters.h`

```cpp
#pragma once
#include <string>
#include <vector>
enum class CoverType
{
    Overlay,
    Inset
};
enum class SealType
{
    Type1,
    Type2
};
struct CoverInput
{
    CoverType coverType = CoverType::Overlay;
    double D = 0.0;
    double h = 0.0;
    bool withSeal = false;
    SealType sealType = SealType::Type1;
    double shaftDiameter = 0.0;
};
struct CoverParameters
{
    CoverType coverType = CoverType::Overlay;
    double D = 0.0;
    double D1 = 0.0;
    double D2 = 0.0;
    double D3 = 0.0;
    double H = 0.0;
    double h = 0.0;
```

```
    double hMin = 0.0;
    double hMax = 0.0;
    double s = 0.0;
    double b = 0.0;
    double holeDiameter = 0.0;
    double counterboreDiameter = 0.0;
    int holeCount = 0;
    std::wstring fastenerThread;
    double outerMargin = 0.0;
    double outerFilletRadius = 0.0;
    bool withSeal = false;
    SealType sealType = SealType::Type1;
    double shaftDiameter = 0.0;
    double sealOuterDiameter = 0.0;
    double sealWidth = 0.0;
    double shaftClearanceDiameter = 0.0;
    double filletRadius = 2.0;
    double rearWall = 5.0;
};
bool CalculateCoverParameters(
    const CoverInput& input,
    CoverParameters& output,
    std::wstring& error);
std::vector<double> GetSealShaftDiameters(SealType type);
```

Файл `CoverParameters.cpp`

```cpp
#include "pch.h"
#include "CoverParameters.h"
#include <algorithm>
#include <cmath>
```

```
namespace
{
    constexpr double kEpsilon = 1e-7;
    struct SealRow
    {
        double shaftDiameter;
        double outerDiameter;
        double h1;
        double h2;
    };
    const SealRow kSealRows[] =
    {
        { 10, 26, 7, 0 }, { 11, 26, 7, 0 },
        { 12, 28, 7, 0 }, { 13, 28, 7, 0 }, { 14, 28, 7, 0 },
        { 15, 30, 7, 0 }, { 16, 30, 7, 0 },
        { 17, 32, 7, 0 },
        { 18, 35, 7, 0 }, { 19, 35, 7, 0 },
        { 20, 40, 10, 14 }, { 21, 40, 10, 14 }, { 22, 40, 10, 14 },
        { 24, 41, 10, 14 }, { 25, 42, 10, 14 }, { 26, 45, 10, 14 },
        { 30, 52, 10, 14 }, { 32, 52, 10, 14 },
        { 35, 58, 10, 14 }, { 36, 58, 10, 14 }, { 38, 58, 10, 14 },
        { 40, 60, 10, 14 }, { 42, 62, 10, 14 }, { 45, 65, 10, 14 },
        { 48, 70, 10, 14 }, { 50, 70, 10, 14 }, { 52, 75, 10, 14 },
        { 55, 80, 12, 16 }, { 56, 80, 12, 16 }, { 58, 80, 12, 16 },
        { 60, 85, 12, 16 },
        { 63, 90, 12, 16 }, { 65, 90, 12, 16 },
        { 70, 95, 12, 16 }, { 71, 95, 12, 16 },
        { 75, 100, 12, 16 }, { 80, 105, 12, 16 }, { 85, 110, 12, 16 },
        { 90, 120, 12, 16 }, { 95, 120, 12, 16 }, { 100, 125, 12, 16 }
    };
    bool InRange(double value, double minimum, double maximum)
    {
        return value + kEpsilon >= minimum && value - kEpsilon <=
```

```
maximum;
    }
    const SealRow* FindSealRow(double shaftDiameter)
    {
        for (const auto& row : kSealRows)
        {
            if (std::abs(row.shaftDiameter - shaftDiameter) <
kEpsilon)
                return &row;
        }
        return nullptr;
    }
    bool CalculateOverlay(const CoverInput& input, CoverParameters& p,
std::wstring& error)
    {
        if (InRange(input.D, 40, 62))
        {
            p.D1 = input.D + 15;
            p.D2 = input.D + 30;
            p.D3 = input.D - 10;
            p.holeDiameter = 7;
            p.counterboreDiameter = 14;
            p.fastenerThread = L"M6";
            p.holeCount = 6;
            p.H = 10;
            p.hMin = 5;
            p.hMax = 15;
            p.s = 5;
        }
        else if (input.D >= 65 - kEpsilon && input.D < 75 - kEpsilon)
        {
            p.D1 = input.D + 20;
            p.D2 = input.D + 40;
```

```
            p.D3 = input.D - 10;
            p.holeDiameter = 9;
            p.counterboreDiameter = 18;
            p.fastenerThread = L"M8";
            p.holeCount = 6;
            p.H = 12;
            p.hMin = 5;
            p.hMax = 20;
            p.s = 6;
        }
        else if (InRange(input.D, 75, 95))
        {
            p.D1 = input.D + 20;
            p.D2 = input.D + 40;
            p.D3 = input.D - 10;
            p.holeDiameter = 9;
            p.counterboreDiameter = 18;
            p.fastenerThread = L"M8";
            p.holeCount = 6;
            p.H = 12;
            p.hMin = 5;
            p.hMax = 20;
            p.s = 6;
        }
        else if (InRange(input.D, 100, 145))
        {
            p.D1 = input.D + 25;
            p.D2 = input.D + 50;
            p.D3 = input.D - 15;
            p.holeDiameter = 11;
            p.counterboreDiameter = 22;
            p.fastenerThread = L"M10";
            p.holeCount = 6;
```

```
            p.H = 15;
            p.hMin = 5;
            p.hMax = 30;
            p.s = 7;
        }
        else if (InRange(input.D, 150, 180))
        {
            p.D1 = input.D + 30;
            p.D2 = input.D + 60;
            p.D3 = input.D - 15;
            p.holeDiameter = 13;
            p.counterboreDiameter = 24;
            p.fastenerThread = L"M12";
            p.holeCount = 6;
            p.H = 18;
            p.hMin = 5;
            p.hMax = 30;
            p.s = 8;
        }
        else if (InRange(input.D, 190, 220))
        {
            p.D1 = input.D + 30;
            p.D2 = input.D + 60;
            p.D3 = input.D - 20;
            p.holeDiameter = 13;
            p.counterboreDiameter = 24;
            p.fastenerThread = L"M12";
            p.holeCount = 6;
            p.H = 18;
            p.hMin = 5;
            p.hMax = 30;
            p.s = 8;
        }
```

```
        else
        {
            error = L"Диаметр D не входит в поддерживаемые диапазоны
накладных крышек.";
            return false;
        }
        if (!InRange(input.h, p.hMin, p.hMax))
        {
            error = L"Размер h должен находиться в табличном диапазоне
для выбранного D.";
            return false;
        }
        // Радиальный остаток металла между цековкой и наружной
кромкой фланца.
        p.outerMargin = (p.D2 - p.D1 - p.counterboreDiameter) / 2.0;
        if (p.outerMargin > 2.0)
            p.outerFilletRadius = 2.0;
        else if (p.outerMargin > 1.0)
            p.outerFilletRadius = 1.0;
        else
            p.outerFilletRadius = 0.5;
        p.h = input.h;
        return true;
    }
    bool CalculateInset(const CoverInput& input, CoverParameters& p,
std::wstring& error)
    {
        if (InRange(input.D, 40, 62))
        {
            p.D1 = input.D + 5;
            p.D2 = input.D - 10;
            p.H = 20;
            p.h = 5;
```

```
            p.s = 5;
            p.b = 3;
        }
        else if (InRange(input.D, 65, 95))
        {
            p.D1 = input.D + 6;
            p.D2 = input.D - 12;
            p.H = 25;
            p.h = 6;
            p.s = 6;
            p.b = 5;
        }
        else if (InRange(input.D, 100, 145))
        {
            p.D1 = input.D + 7;
            p.D2 = input.D - 15;
            p.H = 25;
            p.h = 7;
            p.s = 7;
            p.b = 5;
        }
        else if (InRange(input.D, 150, 220))
        {
            p.D1 = input.D + 8;
            p.D2 = input.D - 15;
            p.H = 25;
            p.h = 8;
            p.s = 8;
            p.b = 5;
        }
        else
        {
            error = L"Диаметр D не входит в поддерживаемые диапазоны
```

```
закладных крышек.";
            return false;
        }
        return true;
    }
}
bool CalculateCoverParameters(
    const CoverInput& input,
    CoverParameters& output,
    std::wstring& error)
{
    error.clear();
    output = CoverParameters{};
    output.coverType = input.coverType;
    output.D = input.D;
    output.withSeal = input.withSeal;
    output.sealType = input.sealType;
    if (!std::isfinite(input.D) || input.D <= 0)
    {
        error = L"Введите положительный диаметр D.";
        return false;
    }
    const bool dimensionsOk = input.coverType == CoverType::Overlay
        ? CalculateOverlay(input, output, error)
        : CalculateInset(input, output, error);
    if (!dimensionsOk)
        return false;
    if (!input.withSeal)
        return true;
    const SealRow* seal = FindSealRow(input.shaftDiameter);
    if (seal == nullptr)
    {
        error = L"Выберите стандартный диаметр вала для манжеты.";
```

```
        return false;
    }
    const double width = input.sealType == SealType::Type1 ? seal->h1
: seal->h2;
    if (width <= 0)
    {
        error = L"Манжета типа 2 для выбранного диаметра вала
отсутствует в таблице.";
        return false;
    }
    output.shaftDiameter = seal->shaftDiameter;
    output.sealOuterDiameter = seal->outerDiameter;
    output.sealWidth = width;
    output.shaftClearanceDiameter = seal->shaftDiameter + 1;
    const double availableDiameter = input.coverType ==
CoverType::Overlay ? output.D3 : output.D2;
    if (output.sealOuterDiameter + 2 * output.filletRadius >
availableDiameter + kEpsilon)
    {
        error = L"Выбранная манжета не помещается в крышку по диаметру
с учётом скругления R2.";
        return false;
    }
    const double axialLength = input.coverType == CoverType::Overlay
        ? output.H + output.h
        : output.H;
    if (output.sealWidth + output.rearWall + output.filletRadius >
axialLength + kEpsilon)
    {
        error = L"Выбранная манжета не помещается в крышку по осевой
длине.";
        return false;
    }
```

````
    return true;
}
std::vector<double> GetSealShaftDiameters(SealType type)
{
    std::vector<double> result;
    for (const auto& row : kSealRows)
    {
        if (type == SealType::Type1 || row.h2 > 0)
            result.push_back(row.shaftDiameter);
    }
    return result;
}
```

## Файл `CKompas.h`

```cpp
#pragma once
#include "CoverParameters.h"
#include <string>
class CKompas
{
public:
    CKompas();
    bool Connect();
    bool IsConnected() const;
    bool CreateCover(const CoverParameters& parameters, std::wstring&
error);
    Kompas6API5::KompasObjectPtr KompasApp5;
    Kompas6API5::ksDocument3DPtr Doc;
    Kompas6API5::ksPartPtr Part;
};
```
````
````

Файл `CKompas.cpp`

```cpp
#include "pch.h"
#include "CKompas.h"
#include <algorithm>
#include <cmath>
namespace
{
    constexpr double kPi = 3.14159265358979323846;
    class ProfileWriter
    {
    public:
        ProfileWriter(const Kompas6API5::ksDocument2DPtr& document,
double startX, double startY)
            : m_document(document), m_startX(startX),
m_startY(startY), m_x(startX), m_y(startY)
        {
        }
        bool LineTo(double x, double y)
        {
            if (!m_ok || m_document->ksLineSeg(m_x, m_y, x, y, 1) ==
0)
            {
                m_ok = false;
                return false;
            }
            m_x = x;
            m_y = y;
            return true;
        }
        bool ArcTo(double centerX, double centerY, double x, double y)
```

```
        {
            const double startAngle = std::atan2(m_y - centerY, m_x -
centerX);
            const double endAngle = std::atan2(y - centerY, x -
centerX);
            double delta = endAngle - startAngle;
            while (delta > kPi)
                delta -= 2 * kPi;
            while (delta < -kPi)
                delta += 2 * kPi;
            const double radius = std::hypot(m_x - centerX, m_y -
centerY);
            const double middleAngle = startAngle + delta / 2.0;
            const double middleX = centerX + radius *
std::cos(middleAngle);
            const double middleY = centerY + radius *
std::sin(middleAngle);
            if (!m_ok || m_document->ksArcBy3Points(m_x, m_y, middleX,
middleY, x, y, 1) == 0)
            {
                m_ok = false;
                return false;
            }
            m_x = x;
            m_y = y;
            return true;
        }
        bool Close()
        {
            return LineTo(m_startX, m_startY);
        }
        bool IsValid() const { return m_ok; }
    private:
```

```
        Kompas6API5::ksDocument2DPtr m_document;
        double m_startX;
        double m_startY;
        double m_x;
        double m_y;
        bool m_ok = true;
    };
    bool DrawOverlayProfile(
        const Kompas6API5::ksDocument2DPtr& sketch,
        const CoverParameters& p)
    {
        const double outerRadius = p.D2 / 2.0;
        const double bodyRadius = p.D / 2.0;
        const double cavityRadius = p.D3 / 2.0;
        const double length = p.H + p.h;
        const double radius = p.filletRadius;
        const double outerFillet = p.outerFilletRadius;
        ProfileWriter profile(sketch, 0, outerRadius - outerFillet);
        profile.ArcTo(outerFillet, outerRadius - outerFillet,
            outerFillet, outerRadius);
        profile.LineTo(p.H, outerRadius);
        profile.LineTo(p.H, bodyRadius);
        profile.LineTo(length, bodyRadius);
        if (!p.withSeal)
        {
            profile.LineTo(length, cavityRadius + radius);
            profile.ArcTo(length - radius, cavityRadius + radius,
length - radius, cavityRadius);
            profile.LineTo(p.s + radius, cavityRadius);
            profile.ArcTo(p.s + radius, cavityRadius - radius, p.s,
cavityRadius - radius);
            profile.LineTo(p.s, 0);
            profile.LineTo(0, 0);
```

```
        }
        else
        {
            const double sealRadius = p.sealOuterDiameter / 2.0;
            const double clearanceRadius = p.shaftClearanceDiameter /
2.0;
            const double chamferLength = 2.0;
            const double chamferRise = chamferLength * std::tan(15.0 *
kPi / 180.0);
            const double seatBack = length - p.sealWidth;
            profile.LineTo(length, sealRadius + chamferRise);
            profile.LineTo(length - chamferLength, sealRadius);
            profile.LineTo(seatBack, sealRadius);
            profile.LineTo(seatBack, cavityRadius);
            profile.LineTo(p.rearWall + radius, cavityRadius);
            profile.ArcTo(p.rearWall + radius, cavityRadius - radius,
                p.rearWall, cavityRadius - radius);
            profile.LineTo(p.rearWall, clearanceRadius);
            profile.LineTo(0, clearanceRadius);
        }
        profile.Close();
        if (sketch->ksLineSeg(-5, 0, length + 5, 0, 3) == 0)
            return false;
        return profile.IsValid();
    }
    bool DrawInsetProfile(
        const Kompas6API5::ksDocument2DPtr& sketch,
        const CoverParameters& p)
    {
        const double mainRadius = p.D / 2.0;
        const double flangeRadius = p.D1 / 2.0;
        const double cavityRadius = p.D2 / 2.0;
        const double radius = p.filletRadius;
```

```
        const double grooveStart = p.H - p.h - p.b;
        const double grooveEnd = p.H - p.h;
        const double grooveDepth = (std::min)(1.0, (std::max)(0.5,
(p.D1 - p.D) / 4.0));
        const double grooveRadius = mainRadius - grooveDepth;
        ProfileWriter profile(sketch, 0, mainRadius);
        profile.LineTo(grooveStart, mainRadius);
        profile.LineTo(grooveStart, grooveRadius);
        profile.LineTo(grooveEnd, grooveRadius);
        profile.LineTo(grooveEnd, flangeRadius);
        profile.LineTo(p.H, flangeRadius);
        if (!p.withSeal)
        {
            profile.LineTo(p.H, cavityRadius + radius);
            profile.ArcTo(p.H - radius, cavityRadius + radius, p.H -
radius, cavityRadius);
            profile.LineTo(p.s + radius, cavityRadius);
            profile.ArcTo(p.s + radius, cavityRadius - radius, p.s,
cavityRadius - radius);
            profile.LineTo(p.s, 0);
            profile.LineTo(0, 0);
        }
        else
        {
            const double sealRadius = p.sealOuterDiameter / 2.0;
            const double clearanceRadius = p.shaftClearanceDiameter /
2.0;
            const double chamferLength = 2.0;
            const double chamferRise = chamferLength * std::tan(15.0 *
kPi / 180.0);
            const double seatBack = p.H - p.sealWidth;
            profile.LineTo(p.H, sealRadius + chamferRise);
            profile.LineTo(p.H - chamferLength, sealRadius);
```

```
            profile.LineTo(seatBack, sealRadius);
            profile.LineTo(seatBack, cavityRadius);
            profile.LineTo(p.rearWall + radius, cavityRadius);
            profile.ArcTo(p.rearWall + radius, cavityRadius - radius,
                p.rearWall, cavityRadius - radius);
            profile.LineTo(p.rearWall, clearanceRadius);
            profile.LineTo(0, clearanceRadius);
        }
        profile.LineTo(0, mainRadius);
        if (sketch->ksLineSeg(-5, 0, p.H + 5, 0, 3) == 0)
            return false;
        return profile.IsValid();
    }
    bool CreateRevolvedBody(
        const Kompas6API5::ksPartPtr& part,
        const CoverParameters& parameters)
    {
        Kompas6API5::ksEntityPtr sketchEntity(part-
>NewEntity(o3d_sketch));
        if (!sketchEntity)
            return false;
        Kompas6API5::ksSketchDefinitionPtr
sketchDefinition(sketchEntity->GetDefinition());
        Kompas6API5::ksEntityPtr plane(part-
>GetDefaultEntity(o3d_planeXOY));
        if (!sketchDefinition || !plane || !sketchDefinition-
>SetPlane(plane) || !sketchEntity->Create())
            return false;
        sketchEntity->Putname(L"Осевой профиль крышки");
        Kompas6API5::ksDocument2DPtr sketch(sketchDefinition-
>BeginEdit());
        if (!sketch)
            return false;
```

```
        const bool drawn = parameters.coverType == CoverType::Overlay
            ? DrawOverlayProfile(sketch, parameters)
            : DrawInsetProfile(sketch, parameters);
        if (!sketchDefinition->EndEdit() || !drawn)
            return false;
        Kompas6API5::ksEntityPtr rotation(part-
>NewEntity(o3d_bossRotated));
        if (!rotation)
            return false;
        Kompas6API5::ksBossRotatedDefinitionPtr definition(rotation-
>GetDefinition());
        if (!definition)
            return false;
        definition->PutdirectionType(dtNormal);
        definition->SetThinParam(VARIANT_FALSE, dtNormal, 0, 0);
        definition->SetSideParam(VARIANT_TRUE, 360.0);
        definition->SetSketch(sketchEntity);
        rotation->Putname(L"Тело вращения крышки");
        return rotation->Create() == VARIANT_TRUE;
    }
    bool CreateHoleCut(
        const Kompas6API5::ksPartPtr& part,
        const CoverParameters& parameters,
        double diameter,
        bool throughAll,
        const wchar_t* name)
    {
        Kompas6API5::ksEntityPtr sketchEntity(part-
>NewEntity(o3d_sketch));
        Kompas6API5::ksSketchDefinitionPtr sketchDefinition(
            sketchEntity ? sketchEntity->GetDefinition() : nullptr);
        Kompas6API5::ksEntityPtr plane(part-
>GetDefaultEntity(o3d_planeYOZ));
```

```
        if (!sketchEntity || !sketchDefinition || !plane ||
            !sketchDefinition->SetPlane(plane) || !sketchEntity-
>Create())
            return false;
        Kompas6API5::ksDocument2DPtr sketch(sketchDefinition-
>BeginEdit());
        if (!sketch)
            return false;
        const double pitchRadius = parameters.D1 / 2.0;
        for (int i = 0; i < parameters.holeCount; ++i)
        {
            const double angle = 2.0 * kPi * i / parameters.holeCount;
            const double x = pitchRadius * std::cos(angle);
            const double y = pitchRadius * std::sin(angle);
            if (sketch->ksCircle(x, y, diameter / 2.0, 1) == 0)
            {
                sketchDefinition->EndEdit();
                return false;
            }
        }
        if (!sketchDefinition->EndEdit())
            return false;
        Kompas6API5::ksEntityPtr cut(part-
>NewEntity(o3d_cutExtrusion));
        Kompas6API5::ksCutExtrusionDefinitionPtr definition(cut ? cut-
>GetDefinition() : nullptr);
        if (!cut || !definition)
            return false;
        definition->SetSketch(sketchEntity);
        definition->SetThinParam(VARIANT_FALSE, dtNormal, 0, 0);
        if (throughAll)
        {
            definition->PutdirectionType(dtBoth);
```

```
            definition->SetSideParam(VARIANT_TRUE, etThroughAll, 0, 0,
VARIANT_FALSE);
            definition->SetSideParam(VARIANT_FALSE, etThroughAll, 0,
0, VARIANT_FALSE);
        }
        else
        {
            definition->PutdirectionType(dtNormal);
            definition->SetSideParam(VARIANT_TRUE, etBlind, 2.0, 0,
VARIANT_FALSE);
        }
        cut->Putname(name);
        return cut->Create() == VARIANT_TRUE;
    }
}
CKompas::CKompas()
{
    Connect();
}
bool CKompas::Connect()
{
    KompasApp5 = nullptr;
    Doc = nullptr;
    Part = nullptr;
    CLSID applicationClsid{};
    HRESULT result = CLSIDFromProgID(L"Kompas.Application.5",
&applicationClsid);
    if (FAILED(result))
        return false;
    IUnknown* applicationUnknown = nullptr;
    result = ::GetActiveObject(applicationClsid, nullptr,
&applicationUnknown);
    if (FAILED(result))
```

```
    {
        // Если КОМПАС не запущен, запускаем локальный COM-сервер.
        result = CoCreateInstance(
            applicationClsid,
            nullptr,
            CLSCTX_LOCAL_SERVER,
            IID_IUnknown,
            reinterpret_cast<void**>(&applicationUnknown));
    }
    if (FAILED(result) || applicationUnknown == nullptr)
        return false;
    try
    {
        KompasApp5 = Kompas6API5::KompasObjectPtr(applicationUnknown);
        KompasApp5->PutVisible(VARIANT_TRUE);
    }
    catch (const _com_error&)
    {
        KompasApp5 = nullptr;
        applicationUnknown->Release();
        return false;
    }
    applicationUnknown->Release();
    return KompasApp5 != nullptr;
}
bool CKompas::IsConnected() const
{
    return KompasApp5 != nullptr;
}
bool CKompas::CreateCover(const CoverParameters& parameters,
std::wstring& error)
{
    error.clear();
```

```
    if (!IsConnected() && !Connect())
    {
        error = L"Не удалось подключиться к КОМПАС-3D.";
        return false;
    }
    try
    {
        Doc = Kompas6API5::ksDocument3DPtr(KompasApp5->Document3D());
        if (!Doc || Doc->Create(VARIANT_FALSE, VARIANT_TRUE) !=
VARIANT_TRUE)
        {
            error = L"КОМПАС-3D не создал новый документ-деталь.";
            return false;
        }
        Doc->SetActive();
        Doc->PutdrawMode(vm_Shaded);
        Doc->PutshadedWireframe(VARIANT_TRUE);
        Part = Kompas6API5::ksPartPtr(Doc->GetPart(pTop_Part));
        if (!Part)
        {
            error = L"Не удалось получить главный компонент новой
детали.";
            return false;
        }
        Part->Putname(parameters.coverType == CoverType::Overlay
            ? L"Крышка подшипника накладная"
            : L"Крышка подшипника закладная");
        if (!CreateRevolvedBody(Part, parameters))
        {
            error = L"Не удалось создать осевой профиль или операцию
вращения крышки.";
            return false;
        }
```

```
        if (parameters.coverType == CoverType::Overlay)
        {
            if (!CreateHoleCut(Part, parameters,
parameters.holeDiameter, true,
                L"Крепёжные отверстия"))
            {
                error = L"Не удалось построить крепёжные отверстия
накладной крышки.";
                return false;
            }
            if (!CreateHoleCut(Part, parameters,
parameters.counterboreDiameter, false,
                L"Цековки глубиной 2 мм"))
            {
                error = L"Не удалось построить цековки накладной
крышки.";
                return false;
            }
        }
        Part->Update();
        Part->RebuildModel();
        Doc->RebuildDocument();
        return true;
    }
    catch (const _com_error& exception)
    {
        error = L"Ошибка API КОМПАС-3D: ";
        const wchar_t* message = exception.ErrorMessage();
        if (message != nullptr)
            error += message;
        return false;
    }
}
```

```

## Файл `kursovaya_detmash.h`

```cpp
#pragma once
#ifndef __AFXWIN_H__
    #error "включить pch.h до включения этого файла в PCH"
#endif
#include "resource.h"
class CkursovayadetmashApp : public CWinApp
{
public:
    CkursovayadetmashApp();
public:
    virtual BOOL InitInstance();
    DECLARE_MESSAGE_MAP()
};
extern CkursovayadetmashApp theApp;
```

Файл `kursovaya_detmash.cpp`

```cpp
#include "pch.h"
#include "framework.h"
#include "kursovaya_detmash.h"
#include "kursovaya_detmashDlg.h"
#include "CKompas.h"
#ifdef _DEBUG
#define new DEBUG_NEW
#endif
BEGIN_MESSAGE_MAP(CkursovayadetmashApp, CWinApp)
```
```

```
    ON_COMMAND(ID_HELP, &CWinApp::OnHelp)
END_MESSAGE_MAP()
CkursovayadetmashApp::CkursovayadetmashApp()
{
    m_dwRestartManagerSupportFlags =
AFX_RESTART_MANAGER_SUPPORT_RESTART;
}
CkursovayadetmashApp theApp;
BOOL CkursovayadetmashApp::InitInstance()
{
    // Координаты элементов диалога масштабируются в OnInitDialog.
Режим Per-Monitor V2
    // нужен, чтобы GetDpiForWindow возвращал реальный DPI монитора, а
Windows не растягивала окно как картинку.

::SetProcessDpiAwarenessContext(DPI_AWARENESS_CONTEXT_PER_MONITOR_AWAR
E_V2);
    INITCOMMONCONTROLSEX InitCtrls;
    InitCtrls.dwSize = sizeof(InitCtrls);
    InitCtrls.dwICC = ICC_WIN95_CLASSES;
    InitCommonControlsEx(&InitCtrls);
    CWinApp::InitInstance();
    if (!AfxOleInit())
    {
        AfxMessageBox(_T("Не удалось инициализировать COM/OLE."));
        return FALSE;
    }
    AfxEnableControlContainer();
    CKompas kompas;
    if (!kompas.IsConnected())
    {
        AfxMessageBox(_T("Не удалось запустить КОМПАС-3D."));
        return FALSE;
```

```
    }
    CShellManager *pShellManager = new CShellManager;

CMFCVisualManager::SetDefaultManager(RUNTIME_CLASS(CMFCVisualManagerWi
ndows));
    SetRegistryKey(_T("Локальные приложения, созданные с помощью
мастера приложений"));
    CkursovayadetmashDlg dlg(kompas);
    m_pMainWnd = &dlg;
    INT_PTR nResponse = dlg.DoModal();
    if (nResponse == IDOK)
    {
    }
    else if (nResponse == IDCANCEL)
    {
    }
    else if (nResponse == -1)
    {
        TRACE(traceAppMsg, 0, "Предупреждение. Не удалось создать
диалоговое окно, поэтому работа приложения неожиданно завершена.\n");
        TRACE(traceAppMsg, 0, "Предупреждение. При использовании
элементов управления MFC для диалогового окна невозможно #define
_AFX_NO_MFC_CONTROLS_IN_DIALOGS.\n");
    }
    if (pShellManager != nullptr)
    {
        delete pShellManager;
    }
#if !defined(_AFXDLL) && !defined(_AFX_NO_MFC_CONTROLS_IN_DIALOGS)
    ControlBarCleanUp();
#endif
    return FALSE;
}
```

```

## Файл `kursovaya_detmashDlg.h`

```cpp
#pragma once
#include "CoverParameters.h"
class CKompas;
class CkursovayadetmashDlg : public CDialog
{
public:
    CkursovayadetmashDlg(CKompas& kompas, CWnd* pParent = nullptr);
    virtual ~CkursovayadetmashDlg();
#ifdef AFX_DESIGN_TIME
    enum { IDD = IDD_KURSOVAYA_DETMASH_DIALOG };
#endif
    protected:
    virtual void DoDataExchange(CDataExchange* pDX);
protected:
    HICON m_hIcon;
    virtual BOOL OnInitDialog();
    afx_msg void OnSysCommand(UINT nID, LPARAM lParam);
    afx_msg void OnPaint();
    afx_msg HCURSOR OnQueryDragIcon();
    afx_msg void OnGetMinMaxInfo(MINMAXINFO* lpMMI);
    afx_msg void OnParametersChanged();
    afx_msg void OnSealExecutionChanged();
    afx_msg void OnSealTypeChanged();
    afx_msg void OnSealDiameterChanged();
    afx_msg void OnBuildModel();
    afx_msg void OnResetParameters();
    afx_msg void OnTabSelectionChanged(NMHDR* pNMHDR, LRESULT*
pResult);
```
```

```
    DECLARE_MESSAGE_MAP()
private:
    CKompas& m_kompas;
    CFont m_uiFont;
    std::vector<HWND> m_parameterControls;
    std::vector<HWND> m_drawingPageControls[3];
    HWND m_drawingPreviews[3]{};
    HWND m_drawingHints[3]{};
    HBITMAP m_drawingBitmaps[3]{};
    CSize m_fixedWindowSize;
    bool m_updatingUi = false;
    void CreateMainTabs();
    void CreateParameterControls();
    void CreateDrawingControls();
    void ShowSelectedTab();
    bool LoadDrawing(int index, const CString& path, bool showError);
    void LoadProjectDrawings();
    void ResetParameters();
    void UpdateInterface();
    void PopulateSealDiameters();
    bool TryReadBaseInput(CoverInput& input) const;
    bool HasCompatibleSeal(const CoverInput& baseInput, SealType type)
const;
    bool SelectCompatibleSeal(const CoverInput& baseInput, SealType
type);
    bool ReadInput(CoverInput& input, CString& error, bool
requireSealSelection = true) const;
    void ShowParameters(const CoverParameters* parameters);
    void SetStatus(const CString& text);
};
```

Файл `kursovaya_detmashDlg.cpp`

```cpp
#include "pch.h"
#include "framework.h"
#include "kursovaya_detmash.h"
#include "kursovaya_detmashDlg.h"
#include "CKompas.h"
#include "afxdialogex.h"
#include <atlimage.h>
#include <cmath>
#ifdef _DEBUG
#define new DEBUG_NEW
#endif
namespace
{
    UINT g_layoutDpi = 96;
    HFONT g_layoutFont = nullptr;
    int g_layoutOffsetY = 0;
    std::vector<HWND>* g_createdControls = nullptr;
    int ScaleLayout(int value)
    {
        return ::MulDiv(value, static_cast<int>(g_layoutDpi), 96);
    }
    HWND CreateControl(HWND parent, DWORD exStyle, LPCWSTR className,
LPCWSTR text,
        DWORD style, int x, int y, int width, int height, int id)
    {
        HWND control = ::CreateWindowExW(
            exStyle, className, text, WS_CHILD | WS_VISIBLE | style,
            ScaleLayout(x), ScaleLayout(y + g_layoutOffsetY),
            ScaleLayout(width), ScaleLayout(height), parent,
            reinterpret_cast<HMENU>(static_cast<INT_PTR>(id)),
            AfxGetInstanceHandle(), nullptr);
```

```
        if (control != nullptr)
        {
            ::SendMessage(control, WM_SETFONT,
                reinterpret_cast<WPARAM>(g_layoutFont != nullptr
                    ? g_layoutFont
                    : ::GetStockObject(DEFAULT_GUI_FONT)), TRUE);
            if (g_createdControls != nullptr)
                g_createdControls->push_back(control);
        }
        return control;
    }
    void CreateLabel(HWND parent, LPCWSTR text, int x, int y, int
width, int height = 18)
    {
        CreateControl(parent, 0, L"STATIC", text, SS_LEFT, x, y,
width, height, -1);
    }
    void CreateReadOnlyEdit(HWND parent, int id, int x, int y, int
width)
    {
        CreateControl(parent, WS_EX_CLIENTEDGE, L"EDIT", L"",
            ES_AUTOHSCROLL | ES_READONLY, x, y, width, 22, id);
    }
    CString FormatNumber(double value)
    {
        CString text;
        if (std::abs(value - std::round(value)) < 1e-7)
            text.Format(L"%.0f", value);
        else
        {
            text.Format(L"%.2f", value);
            text.TrimRight(L'0');
            text.TrimRight(L'.');
```

```
        }
        return text;
    }
}
class CAboutDlg : public CDialogEx
{
public:
    CAboutDlg();
#ifdef AFX_DESIGN_TIME
    enum { IDD = IDD_ABOUTBOX };
#endif
    protected:
    virtual void DoDataExchange(CDataExchange* pDX);
protected:
    DECLARE_MESSAGE_MAP()
};
CAboutDlg::CAboutDlg() : CDialogEx(IDD_ABOUTBOX)
{
}
void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
    CDialogEx::DoDataExchange(pDX);
}
BEGIN_MESSAGE_MAP(CAboutDlg, CDialogEx)
END_MESSAGE_MAP()
CkursovayadetmashDlg::CkursovayadetmashDlg(CKompas& kompas, CWnd*
pParent )
    : CDialog(IDD_KURSOVAYA_DETMASH_DIALOG, pParent)
    , m_kompas(kompas)
{
    m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);
}
CkursovayadetmashDlg::~CkursovayadetmashDlg()
```

```
{
    for (HBITMAP bitmap : m_drawingBitmaps)
    {
        if (bitmap != nullptr)
            ::DeleteObject(bitmap);
    }
}
void CkursovayadetmashDlg::DoDataExchange(CDataExchange* pDX)
{
    CDialog::DoDataExchange(pDX);
}
BEGIN_MESSAGE_MAP(CkursovayadetmashDlg, CDialog)
    ON_WM_SYSCOMMAND()
    ON_WM_PAINT()
    ON_WM_QUERYDRAGICON()
    ON_WM_GETMINMAXINFO()
    ON_BN_CLICKED(IDC_RADIO_OVERLAY,
&CkursovayadetmashDlg::OnParametersChanged)
    ON_BN_CLICKED(IDC_RADIO_INSET,
&CkursovayadetmashDlg::OnParametersChanged)
    ON_BN_CLICKED(IDC_CHECK_SEAL,
&CkursovayadetmashDlg::OnSealExecutionChanged)
    ON_BN_CLICKED(IDC_RADIO_SEAL_TYPE1,
&CkursovayadetmashDlg::OnSealTypeChanged)
    ON_BN_CLICKED(IDC_RADIO_SEAL_TYPE2,
&CkursovayadetmashDlg::OnSealTypeChanged)
    ON_EN_CHANGE(IDC_EDIT_D,
&CkursovayadetmashDlg::OnParametersChanged)
    ON_EN_CHANGE(IDC_EDIT_H,
&CkursovayadetmashDlg::OnParametersChanged)
    ON_CBN_SELCHANGE(IDC_COMBO_SHAFT_D,
&CkursovayadetmashDlg::OnSealDiameterChanged)
    ON_BN_CLICKED(IDC_BUTTON_BUILD,
```

```
&CkursovayadetmashDlg::OnBuildModel)
    ON_BN_CLICKED(IDC_BUTTON_RESET,
&CkursovayadetmashDlg::OnResetParameters)
    ON_NOTIFY(TCN_SELCHANGE, IDC_TAB_MAIN,
&CkursovayadetmashDlg::OnTabSelectionChanged)
END_MESSAGE_MAP()
BOOL CkursovayadetmashDlg::OnInitDialog()
{
    CDialog::OnInitDialog();
    g_layoutDpi = ::GetDpiForWindow(GetSafeHwnd());
    if (g_layoutDpi == 0)
        g_layoutDpi = 96;
    LOGFONT uiFont{};
    uiFont.lfHeight = -::MulDiv(9, static_cast<int>(g_layoutDpi), 72);
    uiFont.lfWeight = FW_NORMAL;
    uiFont.lfCharSet = DEFAULT_CHARSET;
    wcscpy_s(uiFont.lfFaceName, L"Segoe UI");
    m_uiFont.CreateFontIndirect(&uiFont);
    g_layoutFont = static_cast<HFONT>(m_uiFont.GetSafeHandle());
    SetFont(&m_uiFont);
    ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
    ASSERT(IDM_ABOUTBOX < 0xF000);
    CMenu* pSysMenu = GetSystemMenu(FALSE);
    if (pSysMenu != nullptr)
    {
        BOOL bNameValid;
        CString strAboutMenu;
        bNameValid = strAboutMenu.LoadString(IDS_ABOUTBOX);
        ASSERT(bNameValid);
        if (!strAboutMenu.IsEmpty())
        {
            pSysMenu->AppendMenu(MF_SEPARATOR);
            pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX,
```

```
strAboutMenu);
        }
    }
    SetIcon(m_hIcon, TRUE);
    SetIcon(m_hIcon, FALSE);
    SetWindowText(L"Построение крышки подшипника редуктора");
    ModifyStyle(WS_THICKFRAME | WS_MAXIMIZEBOX, 0, SWP_FRAMECHANGED);
    SetWindowPos(nullptr, 0, 0, ScaleLayout(880), ScaleLayout(610),
        SWP_NOMOVE | SWP_NOZORDER | SWP_FRAMECHANGED);
    CRect fixedWindowRect;
    GetWindowRect(&fixedWindowRect);
    m_fixedWindowSize = fixedWindowRect.Size();
    if (CMenu* systemMenu = GetSystemMenu(FALSE))
    {
        systemMenu->RemoveMenu(SC_SIZE, MF_BYCOMMAND);
        systemMenu->RemoveMenu(SC_MAXIMIZE, MF_BYCOMMAND);
        DrawMenuBar();
    }
    CenterWindow();
    if (CWnd* okButton = GetDlgItem(IDOK))
        okButton->ShowWindow(SW_HIDE);
    if (CWnd* placeholder = GetDlgItem(IDC_STATIC))
        placeholder->ShowWindow(SW_HIDE);
    CreateMainTabs();
    g_layoutOffsetY = 34;
    g_createdControls = &m_parameterControls;
    CreateParameterControls();
    g_createdControls = nullptr;
    g_layoutOffsetY = 0;
    CreateDrawingControls();
    LoadProjectDrawings();
    ShowSelectedTab();
    SetDefID(IDC_BUTTON_BUILD);
```

```
    ResetParameters();
    return TRUE;
}
void CkursovayadetmashDlg::OnSysCommand(UINT nID, LPARAM lParam)
{
    if ((nID & 0xFFF0) == IDM_ABOUTBOX)
    {
        CAboutDlg dlgAbout;
        dlgAbout.DoModal();
    }
    else
    {
        CDialog::OnSysCommand(nID, lParam);
    }
}
void CkursovayadetmashDlg::OnPaint()
{
    if (IsIconic())
    {
        CPaintDC dc(this);
        SendMessage(WM_ICONERASEBKGND,
reinterpret_cast<WPARAM>(dc.GetSafeHdc()), 0);
        int cxIcon = GetSystemMetrics(SM_CXICON);
        int cyIcon = GetSystemMetrics(SM_CYICON);
        CRect rect;
        GetClientRect(&rect);
        int x = (rect.Width() - cxIcon + 1) / 2;
        int y = (rect.Height() - cyIcon + 1) / 2;
        dc.DrawIcon(x, y, m_hIcon);
    }
    else
    {
        CDialog::OnPaint();
```

```
    }
}
HCURSOR CkursovayadetmashDlg::OnQueryDragIcon()
{
    return static_cast<HCURSOR>(m_hIcon);
}
void CkursovayadetmashDlg::OnGetMinMaxInfo(MINMAXINFO* lpMMI)
{
    CDialog::OnGetMinMaxInfo(lpMMI);
    if (m_fixedWindowSize.cx <= 0 || m_fixedWindowSize.cy <= 0)
        return;
    lpMMI->ptMinTrackSize.x = m_fixedWindowSize.cx;
    lpMMI->ptMinTrackSize.y = m_fixedWindowSize.cy;
    lpMMI->ptMaxTrackSize.x = m_fixedWindowSize.cx;
    lpMMI->ptMaxTrackSize.y = m_fixedWindowSize.cy;
}
void CkursovayadetmashDlg::CreateMainTabs()
{
    HWND tab = ::CreateControl(
        GetSafeHwnd(), 0, WC_TABCONTROLW, L"",
        TCS_BUTTONS | TCS_FLATBUTTONS | TCS_FIXEDWIDTH | WS_TABSTOP,
        10, 6, 820, 30, IDC_TAB_MAIN);
    if (tab == nullptr)
        return;
    ::SendMessage(tab, TCM_SETITEMSIZE, 0,
        MAKELPARAM(ScaleLayout(195), ScaleLayout(24)));
    const wchar_t* titles[] =
    {
        L"Параметры",
        L"Накладная крышка",
        L"Закладная крышка",
        L"Манжеты"
    };
```

```
    for (int index = 0; index < 4; ++index)
    {
        TCITEM item{};
        item.mask = TCIF_TEXT;
        item.pszText = const_cast<LPWSTR>(titles[index]);
        ::SendMessage(tab, TCM_INSERTITEM, index,
            reinterpret_cast<LPARAM>(&item));
    }
    ::SendMessage(tab, TCM_SETCURSEL, 0, 0);
}
void CkursovayadetmashDlg::CreateDrawingControls()
{
    const wchar_t* hints[] =
    {
        L"Чертёж накладной крышки ещё не выбран",
        L"Чертёж закладной крышки ещё не выбран",
        L"Чертёж манжет ещё не выбран"
    };
    const int previewIds[] =
    {
        IDC_STATIC_DRAWING_OVERLAY,
        IDC_STATIC_DRAWING_INSET,
        IDC_STATIC_DRAWING_SEALS
    };
    const int hintIds[] =
    {
        IDC_STATIC_DRAWING_OVERLAY_HINT,
        IDC_STATIC_DRAWING_INSET_HINT,
        IDC_STATIC_DRAWING_SEALS_HINT
    };
    const wchar_t* sourceNames[] = { L"overlay", L"inset", L"seals" };
    for (int index = 0; index < 3; ++index)
    {
```

```
        g_createdControls = &m_drawingPageControls[index];
        m_drawingPreviews[index] = ::CreateControl(
            GetSafeHwnd(), WS_EX_CLIENTEDGE, L"STATIC", L"",
            SS_BITMAP | SS_CENTERIMAGE | SS_SUNKEN,
            24, 48, 812, 450, previewIds[index]);
        m_drawingHints[index] = ::CreateControl(
            GetSafeHwnd(), 0, L"STATIC", hints[index],
            SS_CENTER | SS_CENTERIMAGE,
            40, 64, 780, 418, hintIds[index]);
        CString sourceLabel;
        sourceLabel.Format(
            L"Файл: drawings\\%s.png (также поддерживаются .jpg, .jpeg
и .bmp)",
            sourceNames[index]);
        ::CreateControl(
            GetSafeHwnd(), 0, L"STATIC", sourceLabel,
            SS_LEFT, 24, 505, 620, 20, -1);
    }
    g_createdControls = nullptr;
}
void CkursovayadetmashDlg::ShowSelectedTab()
{
    CWnd* tabWindow = GetDlgItem(IDC_TAB_MAIN);
    const int selection = tabWindow == nullptr
        ? 0
        : static_cast<int>(tabWindow->SendMessage(TCM_GETCURSEL));
    for (HWND control : m_parameterControls)
        ::ShowWindow(control, selection == 0 ? SW_SHOW : SW_HIDE);
    for (int page = 0; page < 3; ++page)
    {
        const bool visible = selection == page + 1;
        for (HWND control : m_drawingPageControls[page])
            ::ShowWindow(control, visible ? SW_SHOW : SW_HIDE);
```

```
        if (visible && m_drawingBitmaps[page] != nullptr)
            ::ShowWindow(m_drawingHints[page], SW_HIDE);
    }
    SetDefID(selection == 0
        ? IDC_BUTTON_BUILD
        : IDCANCEL);
    Invalidate(FALSE);
}
void CkursovayadetmashDlg::OnTabSelectionChanged(NMHDR*, LRESULT*
pResult)
{
    ShowSelectedTab();
    *pResult = 0;
}
bool CkursovayadetmashDlg::LoadDrawing(
    int index,
    const CString& path,
    bool showError)
{
    if (index < 0 || index >= 3 || m_drawingPreviews[index] ==
nullptr)
        return false;
    CImage source;
    const HRESULT loadResult = source.Load(path);
    if (FAILED(loadResult) || source.GetWidth() <= 0 ||
source.GetHeight() <= 0)
    {
        if (showError)
            AfxMessageBox(L"Не удалось открыть выбранное
изображение.", MB_ICONERROR);
        return false;
    }
    CRect previewRect;
```

```
    ::GetClientRect(m_drawingPreviews[index], &previewRect);
    const int canvasWidth = previewRect.Width();
    const int canvasHeight = previewRect.Height();
    if (canvasWidth <= 0 || canvasHeight <= 0)
        return false;
    CImage canvas;
    if (!canvas.Create(canvasWidth, canvasHeight, 32))
        return false;
    HDC canvasDc = canvas.GetDC();
    RECT background{ 0, 0, canvasWidth, canvasHeight };
    ::FillRect(canvasDc, &background,
        static_cast<HBRUSH>(::GetStockObject(WHITE_BRUSH)));
    ::SetStretchBltMode(canvasDc, HALFTONE);
    const int padding = ScaleLayout(12);
    const double widthScale = static_cast<double>(canvasWidth - 2 *
padding) / source.GetWidth();
    const double heightScale = static_cast<double>(canvasHeight - 2 *
padding) / source.GetHeight();
    const double scale = (std::min)(widthScale, heightScale);
    const int drawingWidth = (std::max)(1,
static_cast<int>(std::lround(source.GetWidth() * scale)));
    const int drawingHeight = (std::max)(1,
static_cast<int>(std::lround(source.GetHeight() * scale)));
    const int drawingX = (canvasWidth - drawingWidth) / 2;
    const int drawingY = (canvasHeight - drawingHeight) / 2;
    source.Draw(canvasDc, drawingX, drawingY, drawingWidth,
drawingHeight);
    canvas.ReleaseDC();
    HBITMAP newBitmap = canvas.Detach();
    ::SendMessage(m_drawingPreviews[index], STM_SETIMAGE,
        IMAGE_BITMAP, reinterpret_cast<LPARAM>(newBitmap));
    if (m_drawingBitmaps[index] != nullptr)
        ::DeleteObject(m_drawingBitmaps[index]);
```

```
    m_drawingBitmaps[index] = newBitmap;
    ::ShowWindow(m_drawingHints[index], SW_HIDE);
    ::InvalidateRect(m_drawingPreviews[index], nullptr, TRUE);
    return true;
}
void CkursovayadetmashDlg::LoadProjectDrawings()
{
    wchar_t modulePath[MAX_PATH]{};
    if (::GetModuleFileNameW(nullptr, modulePath,
_countof(modulePath)) == 0)
        return;
    CString executableDirectory(modulePath);
    const int lastSlash = executableDirectory.ReverseFind(L'\\');
    if (lastSlash < 0)
        return;
    executableDirectory = executableDirectory.Left(lastSlash);
    const wchar_t* baseNames[] = { L"overlay", L"inset", L"seals" };
    const wchar_t* extensions[] = { L".png", L".jpg", L".jpeg",
L".bmp" };
    CString projectDrawingDirectory;
    projectDrawingDirectory.Format(L"%s\\..\\..\\drawings",
        executableDirectory.GetString());
    const CString drawingDirectories[] =
    {
        projectDrawingDirectory,
        executableDirectory + L"\\drawings"
    };
    for (int index = 0; index < 3; ++index)
    {
        bool loaded = false;
        for (const CString& drawingDirectory : drawingDirectories)
        {
            for (const wchar_t* extension : extensions)
```

```
            {
                CString path;
                path.Format(L"%s\\%s%s",
                    drawingDirectory.GetString(), baseNames[index],
extension);
                const DWORD attributes = ::GetFileAttributesW(path);
                if (attributes == INVALID_FILE_ATTRIBUTES ||
                    (attributes & FILE_ATTRIBUTE_DIRECTORY) != 0)
                    continue;
                if (LoadDrawing(index, path, false))
                {
                    loaded = true;
                    break;
                }
            }
            if (loaded)
                break;
        }
    }
}
#define CreateControl(...) ::CreateControl(__VA_ARGS__)
void CkursovayadetmashDlg::CreateParameterControls()
{
    const HWND parent = GetSafeHwnd();
    CreateControl(parent, 0, L"BUTTON", L"Тип крышки", BS_GROUPBOX,
10, 8, 390, 54, -1);
    CreateControl(parent, 0, L"BUTTON", L"Накладная",
BS_AUTORADIOBUTTON | WS_GROUP | WS_TABSTOP,
        24, 29, 130, 20, IDC_RADIO_OVERLAY);
    CreateControl(parent, 0, L"BUTTON", L"Закладная",
BS_AUTORADIOBUTTON | WS_TABSTOP,
        190, 29, 130, 20, IDC_RADIO_INSET);
    CreateControl(parent, 0, L"BUTTON", L"Основные параметры, мм",
```

```
BS_GROUPBOX, 10, 68, 390, 118, -1);
    CreateLabel(parent, L"Исходный диаметр D:", 24, 92, 155);
    CreateControl(parent, WS_EX_CLIENTEDGE, L"EDIT", L"",
ES_AUTOHSCROLL | WS_TABSTOP,
        185, 88, 75, 23, IDC_EDIT_D);
    CreateLabel(parent, L"Размер крышки h:", 24, 124, 155);
    CreateControl(parent, WS_EX_CLIENTEDGE, L"EDIT", L"",
ES_AUTOHSCROLL | WS_TABSTOP,
        185, 120, 75, 23, IDC_EDIT_H);
    CreateControl(parent, 0, L"STATIC", L"Введите D", SS_LEFT,
        270, 124, 116, 18, IDC_STATIC_H_RANGE);
    CreateControl(parent, 0, L"BUTTON", L"Исполнение с манжетой",
BS_AUTOCHECKBOX | WS_TABSTOP,
        24, 154, 210, 20, IDC_CHECK_SEAL);
    CreateControl(parent, 0, L"BUTTON", L"Манжета по ГОСТ 8752-79",
BS_GROUPBOX, 10, 192, 390, 142, -1);
    CreateControl(parent, 0, L"BUTTON", L"Тип 1 — без пыльника",
        BS_AUTORADIOBUTTON | WS_GROUP | WS_TABSTOP, 24, 216, 170, 20,
IDC_RADIO_SEAL_TYPE1);
    CreateControl(parent, 0, L"BUTTON", L"Тип 2 — с пыльником",
        BS_AUTORADIOBUTTON | WS_TABSTOP, 205, 216, 175, 20,
IDC_RADIO_SEAL_TYPE2);
    CreateLabel(parent, L"Диаметр вала d*:", 24, 250, 145);
    CreateControl(parent, WS_EX_CLIENTEDGE, L"COMBOBOX", L"",
        CBS_DROPDOWNLIST | WS_VSCROLL | WS_TABSTOP, 185, 245, 75, 190,
IDC_COMBO_SHAFT_D);
    CreateLabel(parent, L"Посадочный D*:", 24, 282, 125);
    CreateReadOnlyEdit(parent, IDC_EDIT_SEAL_D_RESULT, 150, 278, 75);
    CreateLabel(parent, L"Ширина h*:", 238, 282, 75);
    CreateReadOnlyEdit(parent, IDC_EDIT_SEAL_H_RESULT, 310, 278, 65);
    CreateLabel(parent, L"Проходное отверстие равно d* + 1 мм", 24,
309, 320);
    CreateControl(parent, 0, L"BUTTON", L"Рассчитанные параметры, мм",
```

```
BS_GROUPBOX,
        410, 8, 420, 326, -1);
    CreateLabel(parent, L"D1", 430, 36, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_D1_RESULT, 458, 31, 70);
    CreateLabel(parent, L"D2", 550, 36, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_D2_RESULT, 578, 31, 70);
    CreateLabel(parent, L"D3", 670, 36, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_D3_RESULT, 698, 31, 70);
    CreateLabel(parent, L"H", 430, 70, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_H_RESULT, 458, 65, 70);
    CreateLabel(parent, L"h", 550, 70, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_H_SMALL_RESULT, 578, 65, 70);
    CreateLabel(parent, L"s", 670, 70, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_S_RESULT, 698, 65, 70);
    CreateLabel(parent, L"b", 430, 104, 25);
    CreateReadOnlyEdit(parent, IDC_EDIT_B_RESULT, 458, 99, 70);
    CreateLabel(parent, L"R внутр.", 550, 104, 68);
    CreateReadOnlyEdit(parent, IDC_EDIT_FILLET_RESULT, 620, 99, 48);
    CreateLabel(parent, L"R нар.", 680, 104, 56);
    CreateReadOnlyEdit(parent, IDC_EDIT_OUTER_FILLET_RESULT, 738, 99,
52);
    CreateControl(parent, 0, L"BUTTON", L"Крепёж накладной крышки",
BS_GROUPBOX,
        424, 134, 390, 86, -1);
    CreateLabel(parent, L"d отверстия:", 440, 158, 88);
    CreateReadOnlyEdit(parent, IDC_EDIT_HOLE_D_RESULT, 530, 153, 58);
    CreateLabel(parent, L"d1 цековки:", 602, 158, 85);
    CreateReadOnlyEdit(parent, IDC_EDIT_COUNTERBORE_RESULT, 690, 153,
58);
    CreateLabel(parent, L"Резьба:", 440, 190, 55);
    CreateReadOnlyEdit(parent, IDC_EDIT_THREAD_RESULT, 500, 185, 58);
    CreateLabel(parent, L"Количество n:", 602, 190, 88);
    CreateReadOnlyEdit(parent, IDC_EDIT_HOLE_COUNT_RESULT, 690, 185,
```

```
58);
    CreateControl(parent, 0, L"BUTTON", L"Посадочное место манжеты",
BS_GROUPBOX,
        424, 226, 390, 92, -1);
    CreateLabel(parent, L"Отверстие d*+1:", 440, 252, 112);
    CreateReadOnlyEdit(parent, IDC_EDIT_CLEARANCE_RESULT, 554, 247,
65);
    CreateLabel(parent, L"Фаска:", 642, 252, 54);
    HWND chamfer = CreateControl(parent, WS_EX_CLIENTEDGE, L"EDIT",
L"2x15°",
        ES_AUTOHSCROLL | ES_READONLY, 698, 247, 80, 22, -1);
    if (chamfer != nullptr)
        ::SetWindowTextW(chamfer, L"2x15°");
    CreateLabel(parent, L"Входная фаска 2x15°, задняя стенка 5 мм",
440, 286, 320);
    CreateControl(parent, 0, L"BUTTON", L"Порядок работы",
BS_GROUPBOX, 10, 342, 820, 104, -1);
    CreateLabel(parent, L"1. Выберите тип крышки и введите D.", 24,
366, 380);
    CreateLabel(parent, L"2. Для накладной крышки задайте h в
показанном диапазоне.", 24, 389, 390);
    CreateLabel(parent, L"3. При необходимости включите манжету и
задайте её параметры.", 420, 366, 390);
    CreateLabel(parent, L"4. Нажмите «Построить модель». Будет создана
новая деталь.", 420, 389, 390);
    CreateLabel(parent, L"Материал детали не назначается. Модель
остаётся несохранённой в КОМПАС-3D.", 24, 416, 760);
    CreateControl(parent, 0, L"STATIC", L"Введите исходные
параметры.", SS_CENTER,
        10, 454, 820, 22, IDC_STATIC_STATUS);
    CreateControl(parent, 0, L"BUTTON", L"Построить модель",
BS_DEFPUSHBUTTON | WS_TABSTOP,
        420, 490, 132, 30, IDC_BUTTON_BUILD);
```

```
    CreateControl(parent, 0, L"BUTTON", L"Сбросить параметры",
BS_PUSHBUTTON | WS_TABSTOP,
        560, 490, 142, 30, IDC_BUTTON_RESET);
    if (CWnd* closeButton = GetDlgItem(IDCANCEL))
    {
        closeButton->SetWindowText(L"Закрыть");
        closeButton->SetFont(&m_uiFont);
        closeButton->MoveWindow(
            ScaleLayout(710), ScaleLayout(524),
            ScaleLayout(120), ScaleLayout(30));
        closeButton->ShowWindow(SW_SHOW);
    }
}
#undef CreateControl
void CkursovayadetmashDlg::ResetParameters()
{
    m_updatingUi = true;
    CheckRadioButton(IDC_RADIO_OVERLAY, IDC_RADIO_INSET,
IDC_RADIO_OVERLAY);
    CheckDlgButton(IDC_CHECK_SEAL, BST_UNCHECKED);
    CheckRadioButton(IDC_RADIO_SEAL_TYPE1, IDC_RADIO_SEAL_TYPE2,
IDC_RADIO_SEAL_TYPE1);
    SetDlgItemText(IDC_EDIT_D, L"40");
    SetDlgItemText(IDC_EDIT_H, L"5");
    m_updatingUi = false;
    PopulateSealDiameters();
    UpdateInterface();
}
void CkursovayadetmashDlg::OnResetParameters()
{
    ResetParameters();
}
void CkursovayadetmashDlg::OnParametersChanged()
```

```
{
    if (m_updatingUi)
        return;
    PopulateSealDiameters();
    if (IsDlgButtonChecked(IDC_CHECK_SEAL) == BST_CHECKED)
    {
        CoverInput baseInput;
        if (TryReadBaseInput(baseInput))
        {
            const SealType type =
IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) == BST_CHECKED
                ? SealType::Type2
                : SealType::Type1;
            SelectCompatibleSeal(baseInput, type);
        }
    }
    UpdateInterface();
}
void CkursovayadetmashDlg::OnSealExecutionChanged()
{
    if (m_updatingUi)
        return;
    if (IsDlgButtonChecked(IDC_CHECK_SEAL) != BST_CHECKED)
    {
        PopulateSealDiameters();
        UpdateInterface();
        return;
    }
    CoverInput baseInput;
    if (!TryReadBaseInput(baseInput))
    {
        PopulateSealDiameters();
        UpdateInterface();
```

```
        return;
    }
    const bool hasType1 = HasCompatibleSeal(baseInput,
SealType::Type1);
    const bool hasType2 = HasCompatibleSeal(baseInput,
SealType::Type2);
    if (!hasType1 && !hasType2)
    {
        AfxMessageBox(
            L"Для текущих размеров крышки не помещается ни одна
манжета.\n"
            L"Измените D или h, затем снова включите исполнение с
манжетой.",
            MB_ICONWARNING);
        CheckDlgButton(IDC_CHECK_SEAL, BST_UNCHECKED);
        PopulateSealDiameters();
        UpdateInterface();
        return;
    }
    SealType type = IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) ==
BST_CHECKED
        ? SealType::Type2
        : SealType::Type1;
    if ((type == SealType::Type1 && !hasType1) ||
        (type == SealType::Type2 && !hasType2))
    {
        type = hasType1 ? SealType::Type1 : SealType::Type2;
        CheckRadioButton(
            IDC_RADIO_SEAL_TYPE1,
            IDC_RADIO_SEAL_TYPE2,
            type == SealType::Type1 ? IDC_RADIO_SEAL_TYPE1 :
IDC_RADIO_SEAL_TYPE2);
    }
```

```
    PopulateSealDiameters();
    SelectCompatibleSeal(baseInput, type);
    UpdateInterface();
}
void CkursovayadetmashDlg::OnSealTypeChanged()
{
    if (m_updatingUi)
        return;
    const SealType selectedType =
IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) == BST_CHECKED
        ? SealType::Type2
        : SealType::Type1;
    PopulateSealDiameters();
    if (IsDlgButtonChecked(IDC_CHECK_SEAL) != BST_CHECKED)
    {
        UpdateInterface();
        return;
    }
    CoverInput baseInput;
    if (!TryReadBaseInput(baseInput))
    {
        UpdateInterface();
        return;
    }
    if (!HasCompatibleSeal(baseInput, selectedType))
    {
        const CString typeName = selectedType == SealType::Type1 ?
L"1" : L"2";
        CString message;
        message.Format(
            L"Манжеты типа %s не помещаются в крышку с текущими
размерами.\n"
            L"Измените D или h либо выберите другой тип манжеты.",
```

```
            typeName.GetString());
        AfxMessageBox(message, MB_ICONWARNING);
        const SealType otherType = selectedType == SealType::Type1
            ? SealType::Type2
            : SealType::Type1;
        if (HasCompatibleSeal(baseInput, otherType))
        {
            CheckRadioButton(
                IDC_RADIO_SEAL_TYPE1,
                IDC_RADIO_SEAL_TYPE2,
                otherType == SealType::Type1 ? IDC_RADIO_SEAL_TYPE1 :
IDC_RADIO_SEAL_TYPE2);
            PopulateSealDiameters();
            SelectCompatibleSeal(baseInput, otherType);
        }
        else
        {
            CheckDlgButton(IDC_CHECK_SEAL, BST_UNCHECKED);
        }
    }
    else
    {
        SelectCompatibleSeal(baseInput, selectedType);
    }
    UpdateInterface();
}
void CkursovayadetmashDlg::OnSealDiameterChanged()
{
    if (m_updatingUi)
        return;
    CoverInput baseInput;
    if (IsDlgButtonChecked(IDC_CHECK_SEAL) != BST_CHECKED ||
        !TryReadBaseInput(baseInput))
```

```
    {
        UpdateInterface();
        return;
    }
    CComboBox* combo =
reinterpret_cast<CComboBox*>(GetDlgItem(IDC_COMBO_SHAFT_D));
    const int selection = combo == nullptr ? CB_ERR : combo-
>GetCurSel();
    if (selection != CB_ERR)
    {
        CoverInput trial = baseInput;
        trial.withSeal = true;
        trial.sealType = IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) ==
BST_CHECKED
            ? SealType::Type2
            : SealType::Type1;
        trial.shaftDiameter = static_cast<double>(combo-
>GetItemData(selection));
        CoverParameters parameters;
        std::wstring error;
        if (!CalculateCoverParameters(trial, parameters, error))
        {
            AfxMessageBox(CString(error.c_str()), MB_ICONWARNING);
            SelectCompatibleSeal(baseInput, trial.sealType);
        }
    }
    UpdateInterface();
}
bool CkursovayadetmashDlg::ReadInput(
    CoverInput& input,
    CString& error,
    bool requireSealSelection) const
{
```

```
    auto readNumber = [this](int id, double& value) -> bool
    {
        CString text;
        GetDlgItemText(id, text);
        text.Trim();
        text.Replace(L',', L'.');
        if (text.IsEmpty())
            return false;
        wchar_t* end = nullptr;
        value = std::wcstod(text.GetString(), &end);
        return end != text.GetString() && end != nullptr && *end ==
L'\0' && std::isfinite(value);
    };
    input.coverType = IsDlgButtonChecked(IDC_RADIO_INSET) ==
BST_CHECKED
        ? CoverType::Inset
        : CoverType::Overlay;
    if (!readNumber(IDC_EDIT_D, input.D))
    {
        error = L"Введите числовое значение D.";
        return false;
    }
    if (input.coverType == CoverType::Overlay)
    {
        if (!readNumber(IDC_EDIT_H, input.h))
        {
            error = L"Введите числовое значение h.";
            return false;
        }
    }
    else
    {
        input.h = 5;
```

```
    }
    input.withSeal = IsDlgButtonChecked(IDC_CHECK_SEAL) ==
BST_CHECKED;
    input.sealType = IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) ==
BST_CHECKED
        ? SealType::Type2
        : SealType::Type1;
    if (input.withSeal)
    {
        const CComboBox* combo = reinterpret_cast<const
CComboBox*>(GetDlgItem(IDC_COMBO_SHAFT_D));
        const int selection = combo == nullptr ? CB_ERR : combo-
>GetCurSel();
        if (selection == CB_ERR)
        {
            if (!requireSealSelection)
            {
                input.withSeal = false;
                return true;
            }
            error = L"Выберите диаметр вала для манжеты.";
            return false;
        }
        input.shaftDiameter = static_cast<double>(combo-
>GetItemData(selection));
    }
    return true;
}
void CkursovayadetmashDlg::PopulateSealDiameters()
{
    CComboBox* combo =
reinterpret_cast<CComboBox*>(GetDlgItem(IDC_COMBO_SHAFT_D));
    if (combo == nullptr)
```

```
        return;
    int previousValue = -1;
    const int oldSelection = combo->GetCurSel();
    if (oldSelection != CB_ERR)
        previousValue = static_cast<int>(combo-
>GetItemData(oldSelection));
    m_updatingUi = true;
    combo->ResetContent();
    const SealType type = IsDlgButtonChecked(IDC_RADIO_SEAL_TYPE2) ==
BST_CHECKED
        ? SealType::Type2
        : SealType::Type1;
    for (double shaftDiameter : GetSealShaftDiameters(type))
    {
        const CString value = FormatNumber(shaftDiameter);
        const int index = combo->AddString(value);
        combo->SetItemData(index,
static_cast<DWORD_PTR>(std::lround(shaftDiameter)));
        if (static_cast<int>(std::lround(shaftDiameter)) ==
previousValue)
            combo->SetCurSel(index);
    }
    if (combo->GetCurSel() == CB_ERR && combo->GetCount() > 0)
        combo->SetCurSel(0);
    m_updatingUi = false;
}
bool CkursovayadetmashDlg::TryReadBaseInput(CoverInput& input) const
{
    CString inputError;
    if (!ReadInput(input, inputError, false))
        return false;
    input.withSeal = false;
    CoverParameters parameters;
```

```
    std::wstring calculationError;
    return CalculateCoverParameters(input, parameters,
calculationError);
}
bool CkursovayadetmashDlg::HasCompatibleSeal(
    const CoverInput& baseInput,
    SealType type) const
{
    for (double shaftDiameter : GetSealShaftDiameters(type))
    {
        CoverInput trial = baseInput;
        trial.withSeal = true;
        trial.sealType = type;
        trial.shaftDiameter = shaftDiameter;
        CoverParameters parameters;
        std::wstring error;
        if (CalculateCoverParameters(trial, parameters, error))
            return true;
    }
    return false;
}
bool CkursovayadetmashDlg::SelectCompatibleSeal(
    const CoverInput& baseInput,
    SealType type)
{
    CComboBox* combo =
reinterpret_cast<CComboBox*>(GetDlgItem(IDC_COMBO_SHAFT_D));
    if (combo == nullptr)
        return false;
    auto isCompatible = [&baseInput, type](double shaftDiameter) ->
bool
    {
        CoverInput trial = baseInput;
```

```
        trial.withSeal = true;
        trial.sealType = type;
        trial.shaftDiameter = shaftDiameter;
        CoverParameters parameters;
        std::wstring error;
        return CalculateCoverParameters(trial, parameters, error);
    };
    const int currentSelection = combo->GetCurSel();
    if (currentSelection != CB_ERR &&
        isCompatible(static_cast<double>(combo-
>GetItemData(currentSelection))))
        return true;
    for (int index = 0; index < combo->GetCount(); ++index)
    {
        if (isCompatible(static_cast<double>(combo-
>GetItemData(index))))
        {
            combo->SetCurSel(index);
            return true;
        }
    }
    return false;
}
void CkursovayadetmashDlg::UpdateInterface()
{
    if (m_updatingUi)
        return;
    m_updatingUi = true;
    const bool overlay = IsDlgButtonChecked(IDC_RADIO_INSET) !=
BST_CHECKED;
    const bool withSeal = IsDlgButtonChecked(IDC_CHECK_SEAL) ==
BST_CHECKED;
    if (CWnd* hEdit = GetDlgItem(IDC_EDIT_H))
```

```
        hEdit->EnableWindow(overlay);
    for (const int id : { IDC_RADIO_SEAL_TYPE1, IDC_RADIO_SEAL_TYPE2,
IDC_COMBO_SHAFT_D })
    {
        if (CWnd* control = GetDlgItem(id))
            control->EnableWindow(withSeal);
    }
    CoverInput input;
    CString inputError;
    if (!ReadInput(input, inputError))
    {
        ShowParameters(nullptr);
        SetStatus(inputError);
        m_updatingUi = false;
        return;
    }
    CoverInput baseInput = input;
    baseInput.withSeal = false;
    if (baseInput.coverType == CoverType::Overlay && baseInput.h <= 0)
        baseInput.h = 5;
    CoverParameters baseParameters;
    std::wstring baseError;
    if (CalculateCoverParameters(baseInput, baseParameters,
baseError))
    {
        if (overlay)
        {
            CString range;
            range.Format(L"Допустимо: %.0f–%.0f", baseParameters.hMin,
baseParameters.hMax);
            SetDlgItemText(IDC_STATIC_H_RANGE, range);
        }
        else
```

```
        {
            SetDlgItemText(IDC_EDIT_H,
FormatNumber(baseParameters.h));
            SetDlgItemText(IDC_STATIC_H_RANGE, L"По таблице");
        }
    }
    else
    {
        SetDlgItemText(IDC_STATIC_H_RANGE, L"Проверьте D");
    }
    CoverParameters parameters;
    std::wstring calculationError;
    if (CalculateCoverParameters(input, parameters, calculationError))
    {
        ShowParameters(&parameters);
        SetStatus(L"Параметры корректны. Модель готова к
построению.");
    }
    else
    {
        ShowParameters(baseError.empty() ? &baseParameters : nullptr);
        SetStatus(CString(calculationError.c_str()));
    }
    m_updatingUi = false;
}
void CkursovayadetmashDlg::ShowParameters(const CoverParameters* p)
{
    const int resultIds[] =
    {
        IDC_EDIT_D1_RESULT, IDC_EDIT_D2_RESULT, IDC_EDIT_D3_RESULT,
        IDC_EDIT_H_RESULT, IDC_EDIT_H_SMALL_RESULT, IDC_EDIT_S_RESULT,
        IDC_EDIT_B_RESULT, IDC_EDIT_HOLE_D_RESULT,
IDC_EDIT_COUNTERBORE_RESULT,
```

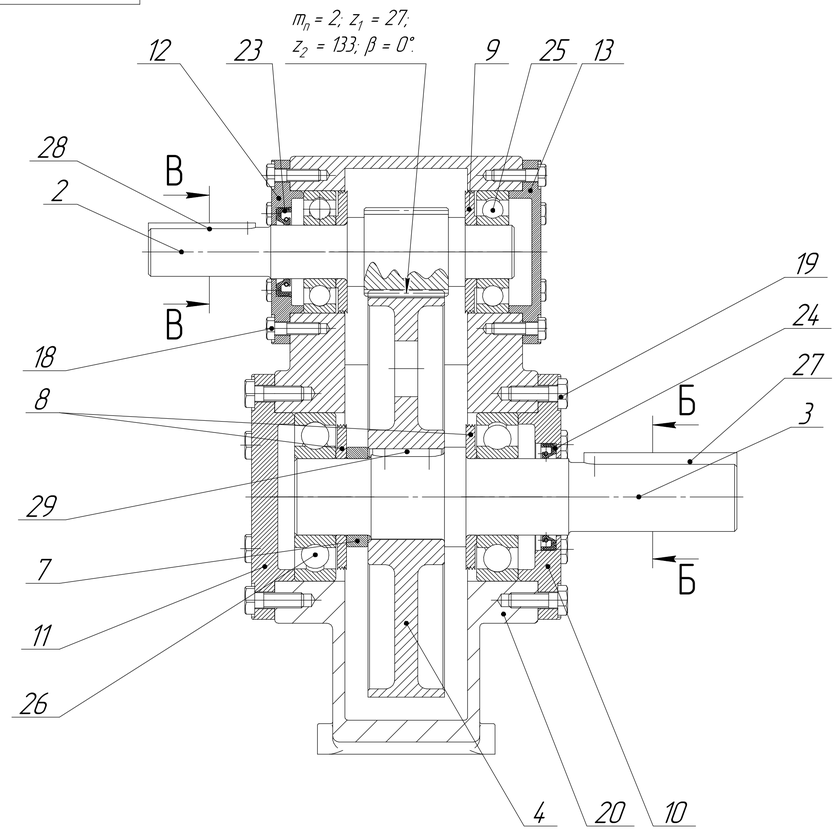
```
        IDC_EDIT_THREAD_RESULT, IDC_EDIT_HOLE_COUNT_RESULT,
        IDC_EDIT_SEAL_D_RESULT, IDC_EDIT_SEAL_H_RESULT,
        IDC_EDIT_CLEARANCE_RESULT, IDC_EDIT_FILLET_RESULT,
        IDC_EDIT_OUTER_FILLET_RESULT
    };
    for (int id : resultIds)
        SetDlgItemText(id, L"");
    if (p == nullptr)
        return;
    SetDlgItemText(IDC_EDIT_D1_RESULT, FormatNumber(p->D1));
    SetDlgItemText(IDC_EDIT_D2_RESULT, FormatNumber(p->D2));
    if (p->coverType == CoverType::Overlay)
        SetDlgItemText(IDC_EDIT_D3_RESULT, FormatNumber(p->D3));
    SetDlgItemText(IDC_EDIT_H_RESULT, FormatNumber(p->H));
    SetDlgItemText(IDC_EDIT_H_SMALL_RESULT, FormatNumber(p->h));
    SetDlgItemText(IDC_EDIT_S_RESULT, FormatNumber(p->s));
    if (p->coverType == CoverType::Inset)
        SetDlgItemText(IDC_EDIT_B_RESULT, FormatNumber(p->b));
    SetDlgItemText(IDC_EDIT_FILLET_RESULT, FormatNumber(p-
>filletRadius));
    if (p->coverType == CoverType::Overlay)
        SetDlgItemText(IDC_EDIT_OUTER_FILLET_RESULT, FormatNumber(p-
>outerFilletRadius));
    if (p->coverType == CoverType::Overlay)
    {
        SetDlgItemText(IDC_EDIT_HOLE_D_RESULT, FormatNumber(p-
>holeDiameter));
        SetDlgItemText(IDC_EDIT_COUNTERBORE_RESULT, FormatNumber(p-
>counterboreDiameter));
        SetDlgItemText(IDC_EDIT_THREAD_RESULT, CString(p-
>fastenerThread.c_str()));
        CString count;
        count.Format(L"%d", p->holeCount);
```

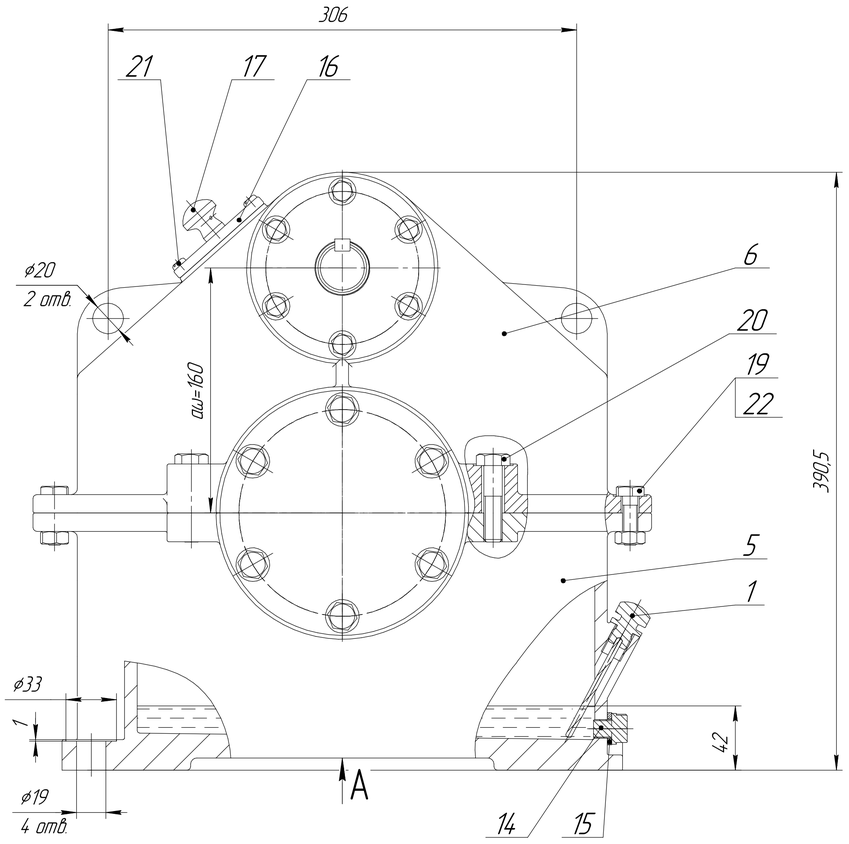
```
        SetDlgItemText(IDC_EDIT_HOLE_COUNT_RESULT, count);
    }
    if (p->withSeal)
    {
        SetDlgItemText(IDC_EDIT_SEAL_D_RESULT, FormatNumber(p-
>sealOuterDiameter));
        SetDlgItemText(IDC_EDIT_SEAL_H_RESULT, FormatNumber(p-
>sealWidth));
        SetDlgItemText(IDC_EDIT_CLEARANCE_RESULT, FormatNumber(p-
>shaftClearanceDiameter));
    }
}
void CkursovayadetmashDlg::SetStatus(const CString& text)
{
    SetDlgItemText(IDC_STATIC_STATUS, text);
}
void CkursovayadetmashDlg::OnBuildModel()
{
    CoverInput input;
    CString inputError;
    if (!ReadInput(input, inputError))
    {
        AfxMessageBox(inputError, MB_ICONWARNING);
        return;
    }
    CoverParameters parameters;
    std::wstring calculationError;
    if (!CalculateCoverParameters(input, parameters,
calculationError))
    {
        AfxMessageBox(CString(calculationError.c_str()),
MB_ICONWARNING);
        return;
```

```
    }
    SetStatus(L"Создание новой детали в КОМПАС-3D...");
    UpdateWindow();
    std::wstring buildError;
    if (!m_kompas.CreateCover(parameters, buildError))
    {
        const CString message = buildError.empty()
            ? L"Не удалось построить модель крышки."
            : CString(buildError.c_str());
        SetStatus(message);
        AfxMessageBox(message, MB_ICONERROR);
        return;
    }
    SetStatus(L"Модель успешно построена в новой несохранённой
детали.");
}
```

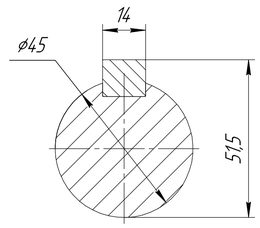
ПРИЛОЖЕНИЕ Б

МПСТ 22.00.00 СБ

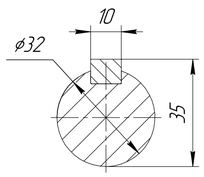


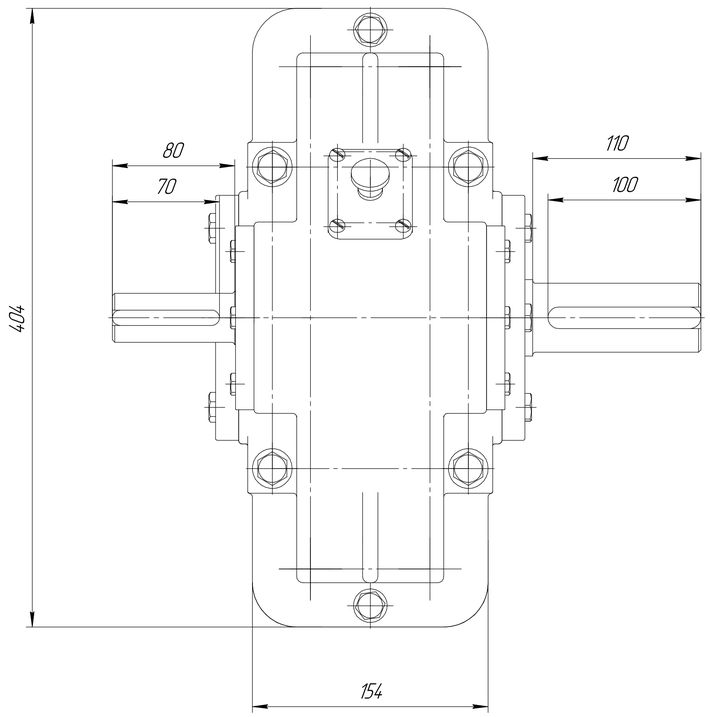


Б–Б(1:1)

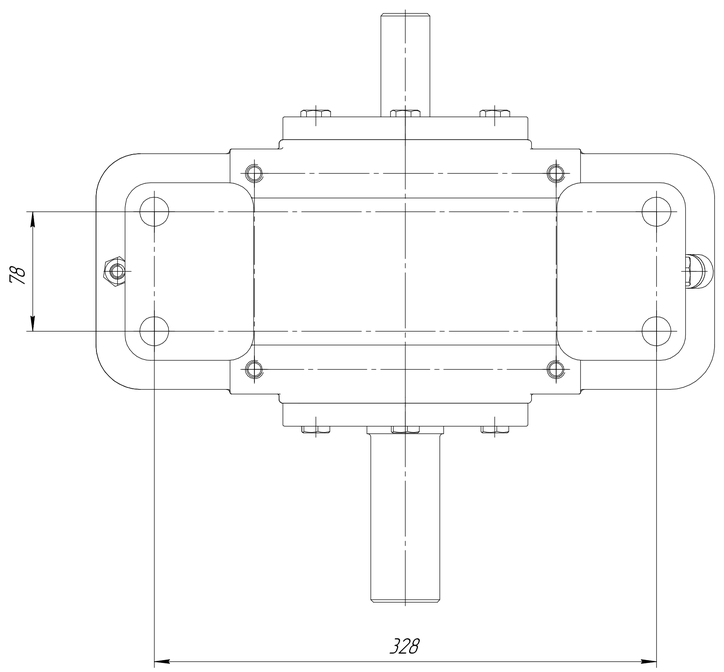


В–В(1:1)





A(1:2)



Техничсекие характеристики

1 Передаточное число редуктора u = 5.
2 Вращающий момент на тихоходном валу T = 244 Н.м.
3 Частота вращения тихоходного вала n = 143,2 об/мин.
4 Мощность на тихоходном валу P = 366.3 Вт.

Технические требования

1 Необработанные поверхности основания и крышки покрасить внутри маслостойкой краской, а снаружи нитроэмалью.
2 Перед сборкой покрыть стыки фланцев корпуса пастой "Герметик".
3 В редукотор залить 0.6 л масла марки И-20А.
4 Ведущий вал должен прокручиваться от руки без заеданий.

					МПСТ 22.00.00 СБ		
Изм.	Лист	№ докум.	Подп.	Дата	Редуктор одноступенчатый прямозубый вертикальный Сборочный чертеж	Лит.	Масса / Масштаб
Разраб.		Фомичева Э.Д.					61.06 / 1:2
Пров.		Леонтьев А.В.				Лист	Листов 1
Т.контр.		Толстиков А.В.					
Н.контр.							
Утв.							

Копировал Формат А1

Перв. примен.
Справ. №
Подп. и дата
Инв. № дубл.
Взам. инв. №
Подп. и дата
Инв. № подл.

КОМПАС-3D v24 Учебная версия © 2025 ООО "АСКОН-Системы проектирования". Россия. Все права защищены.
Не для коммерческого использования

Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Приме-чание
				Документация		
A1			МПСТ 22.00.00 СБ	Сборочный чертёж		
				Сборочные единицы		
		1	МПСТ 22.01.00	Маслощуп	1	
				Детали		
		2	МПСТ 22.00.01	Быстроходный вал-шестерня	1	
		3	МПСТ 22.00.02	Тихоходный вал	1	
		4	МПСТ 22.00.03	Зубчатое колесо	1	
		5	МПСТ 22.00.04	Основание корпуса	1	
		6	МПСТ 22.00.05	Крышка корпуса	1	
		7	МПСТ 22.00.06	Распорная втулка	1	
		8	МПСТ 22.00.07	Мазеудрживающее кольцо	2	
		9	МПСТ 22.00.08	Мазеудрживающее кольцо	2	
		10	МПСТ 22.00.09	Крышка подшипника	1	
		11	МПСТ 22.00.10	Крышка подшипника	1	
		12	МПСТ 22.00.11	Крышка подшипника	1	
		13	МПСТ 22.00.12	Крышка подшипника	1	
		14	МПСТ 22.00.13	Пробка сливная	1	
		15	МПСТ 22.00.14	Уплонение на сливную пробку	1	
		16	МПСТ 22.00.15	Крышка для смотрового отверстия	1	
		17	МПСТ 22.00.16	Отдушина	1	

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.		Фомичёва Э. Д.		
Пров.		Толстиков А. В.		
Н.контр.				
Утв.				

МПСТ 22.00.00 СП

Редуктор прямозубый одноступенчатый вертикальный

Лит.	Лист	Листов
	1	2

КОМПАС-3D v24 Учебная версия © 2025 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Не для коммерческого использования

Копировал

Формат А4

Формат	Зона	Поз.	Обозначение	Наименование	Кол.	Приме-чание
				Стандартные изделия		
		18		Болт М8-6gх30 ГОСТ 7798-70	12	
		19		Болт М10-6gх32 ГОСТ 7798-70	14	
		20		Болт М12-6gх50 ГОСТ 7798-70	4	
		21		Винт А.М5-6gх8 ГОСТ 11644-75	4	
		22		Гайка М10-6Н ГОСТ 5915-70	2	
		23		Манжета 1.1-35 х 58-1 ГОСТ 8752-79	1	
		24		Манжета 1.1-50 х 70-1 ГОСТ 8752-79	1	
		25		Подшипник 307 ГОСТ 520-2011	2	
		26		Подшипник 310 ГОСТ 520-2011	2	
		27		Шпонка 14 х 12 х 100 ГОСТ 10748-79	1	
		28		Шпонка 10 х 8 х 70 ГОСТ 23360-78	1	
		29		Шпонка 16 х 10 х 45 ГОСТ 23360-78	1	
				Материалы		
		32		Масло И-20А ГОСТ 20799-88	0,6	л

КОМПАС-3D v24 Учебная версия © 2025 ООО "АСКОН-Системы проектирования", Россия. Все права защищены.

Инв. № подл. | Подп. и дата | Взам. инв. № | Инв. № дубл. | Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата

МПСТ 22.00.00 СП

Лист 2

Не для коммерческого использования

Копировал

Формат А4